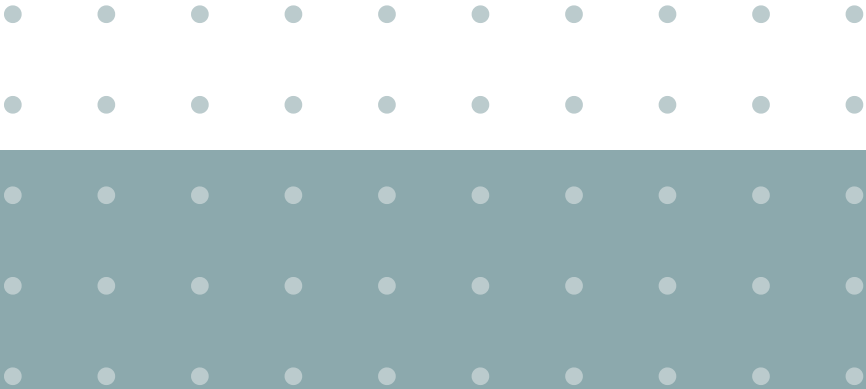




HDFS and MapReduce

Dr. Sirasit Lochanachit

06026213 BIG DATA SYSTEMS

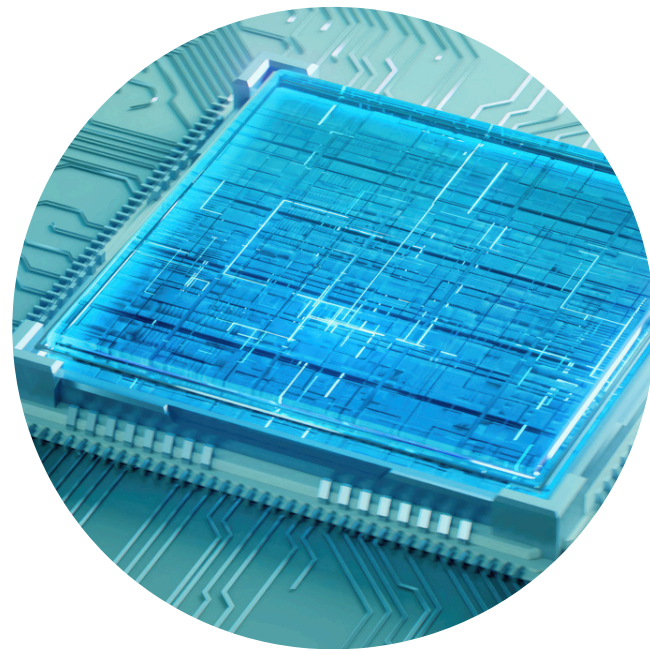




Storing

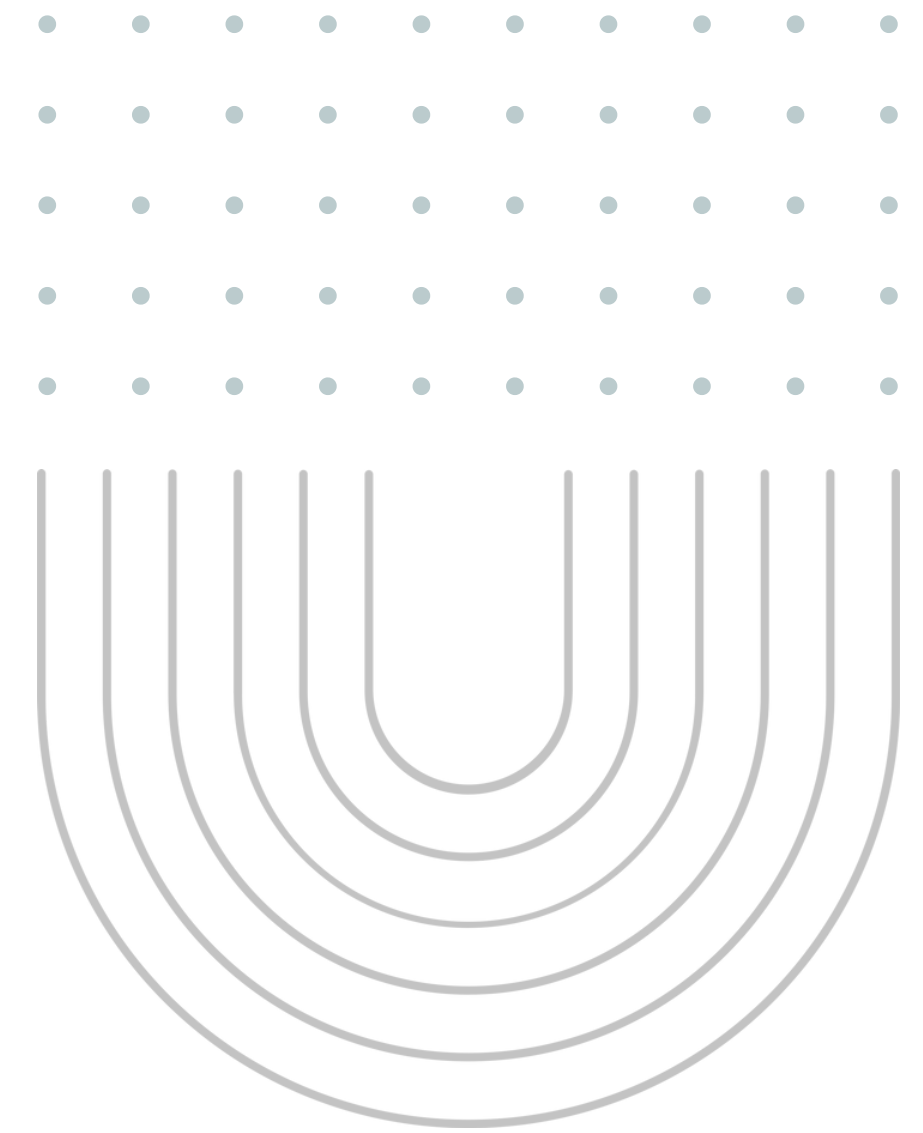
Volume: Massive volume of data

Variety: Images, logs, texts



Processing

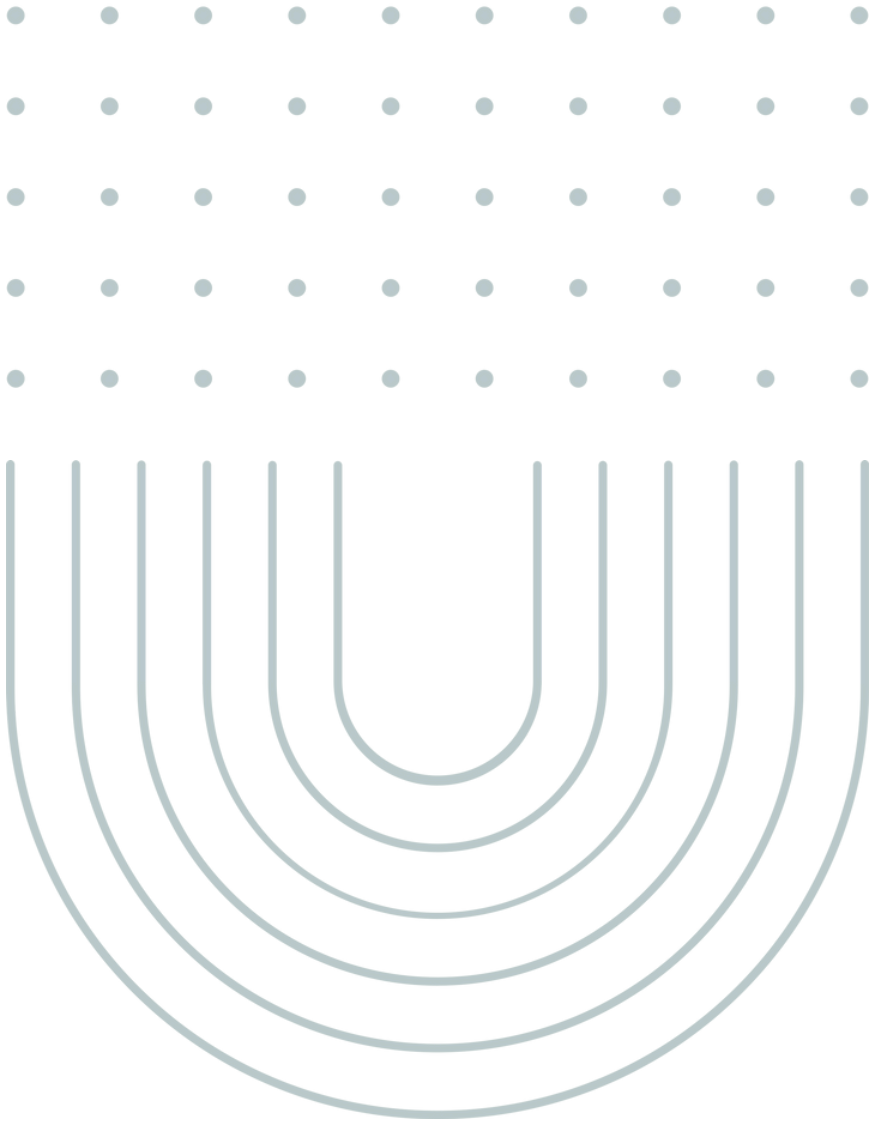
Velocity: Data generated in real-time from diverse sources



BIG DATA CHALLENGES

Others: Fault Tolerance, Scalability, Cost Efficiency

“A collection of open-source tools and **frameworks** designed to facilitate the storage, processing, and analysis of big data across clusters of computers.”

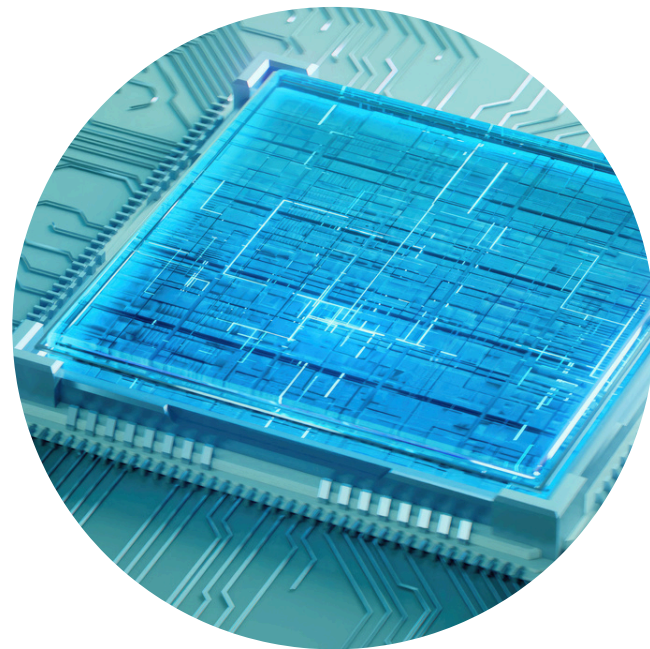


HADOOP ECOSYSTEM



HDFS

Distributed storage system that splits and replicates large datasets across a cluster of nodes.



MapReduce

A programming model for processing large datasets in parallel.

HADOOP ECOSYSTEM

Other Tools: HBase (NoSQL), Hive (DW), Spark (Data Analytics)

01. HDFS

02. MapReduce

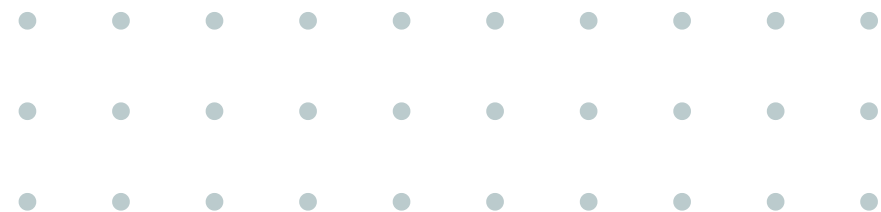


TABLE OF CONTENT

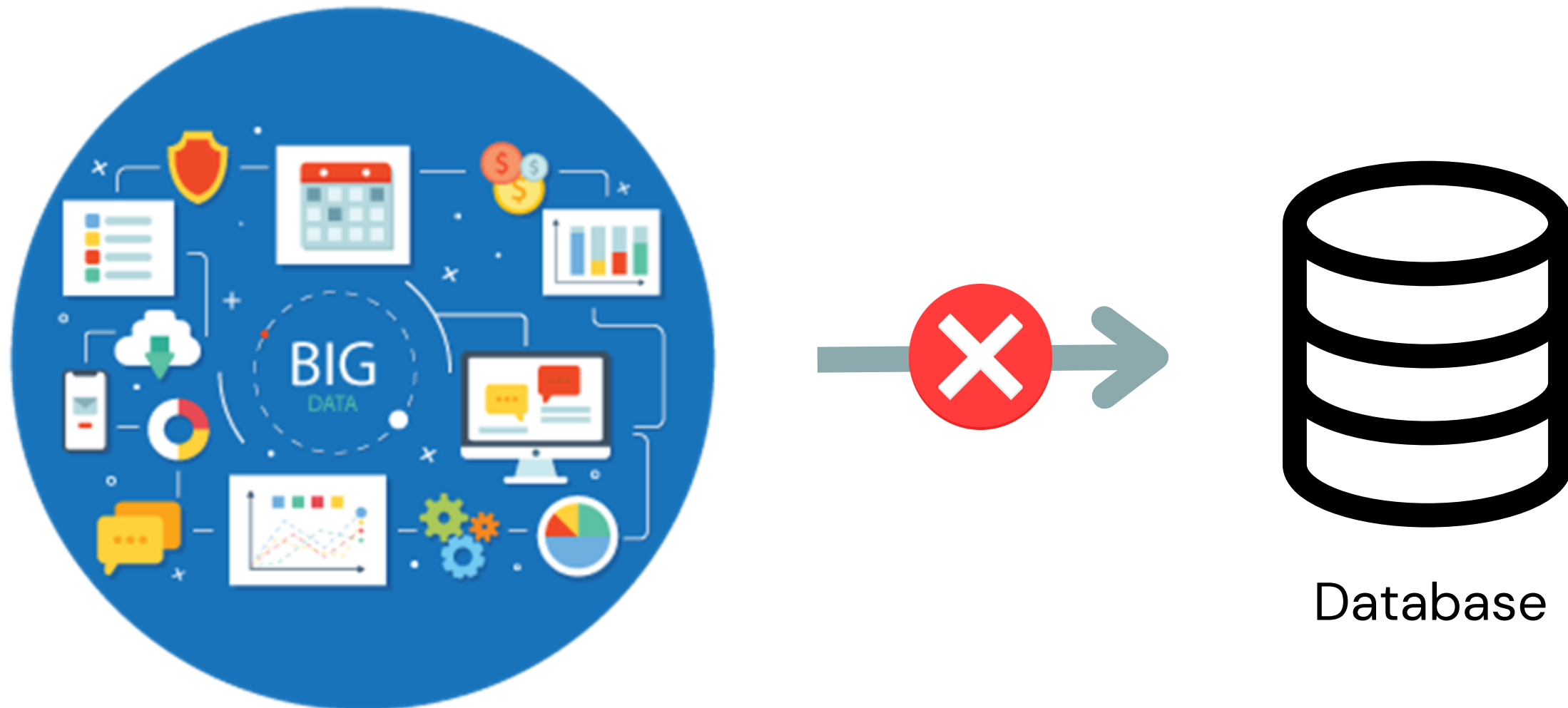
01.

HADOOP DISTRIBUTED FILE SYSTEM (HDFS)

06026213 BIG DATA SYSTEMS

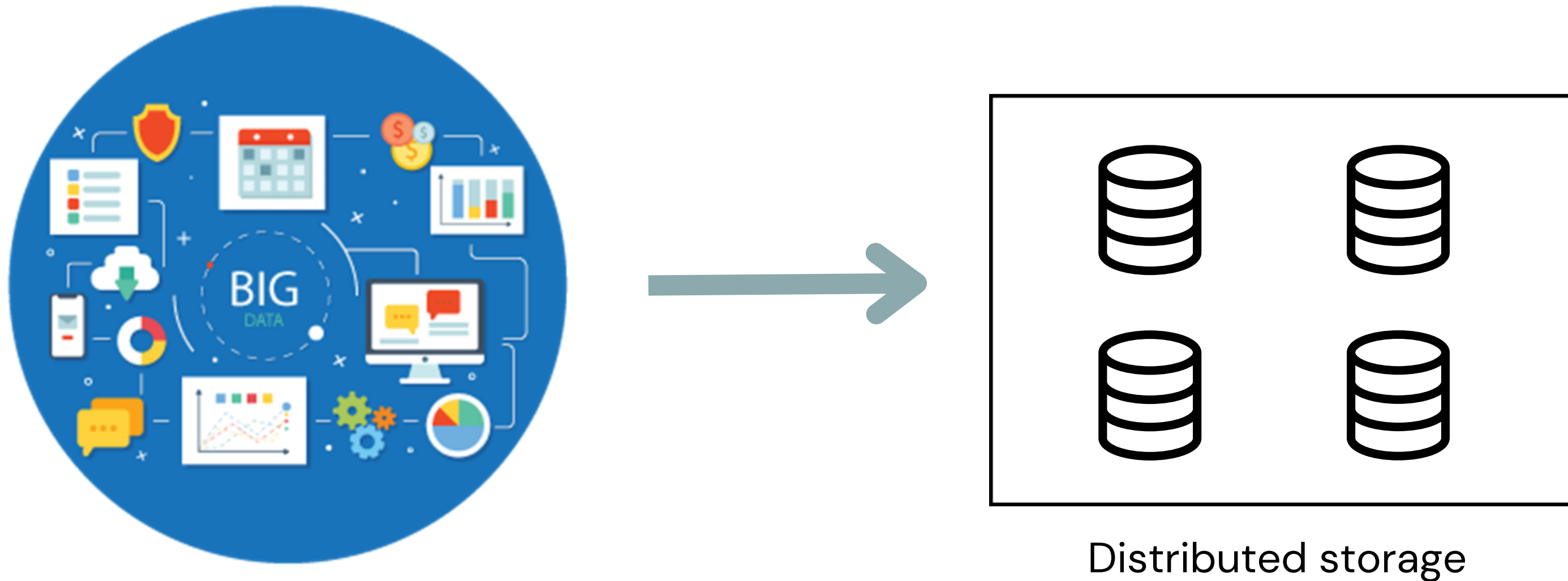
WHY HDFS?

A single central database is not sufficient to store big data



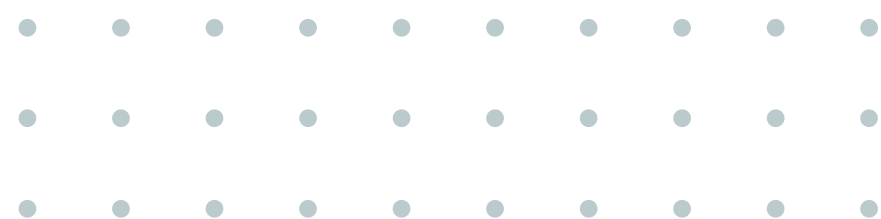
WHY HDFS?

Data can be divided and distributed to individual databases



WHAT IS HDFS?

- A **distributed** file system designed to store large datasets across multiple machines
- Fault-tolerant and designed for low-cost, commodity hardware
- Provides high-throughput access to application data

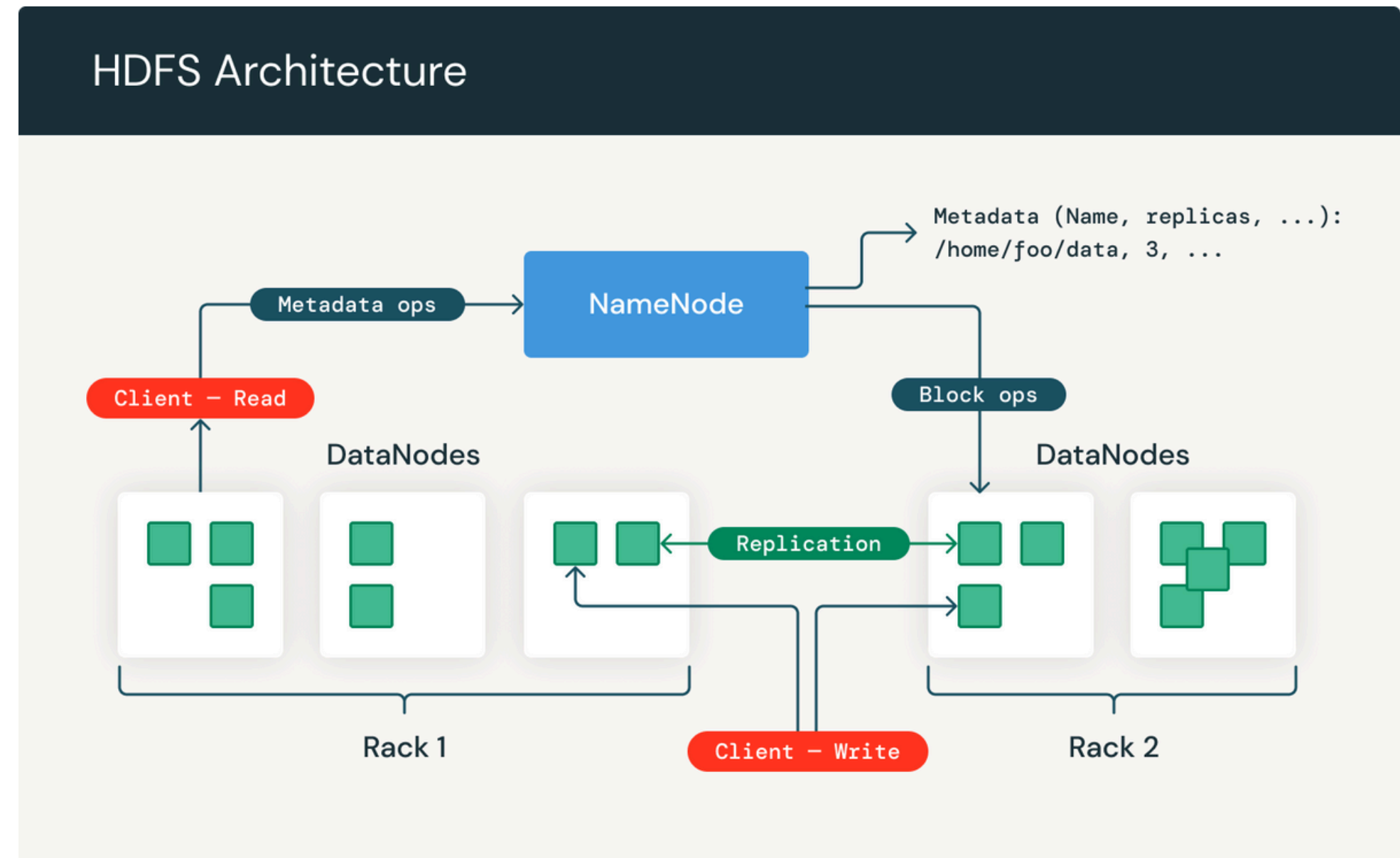


HDFS ARCHITECTURE

2 Core components

1. NameNode

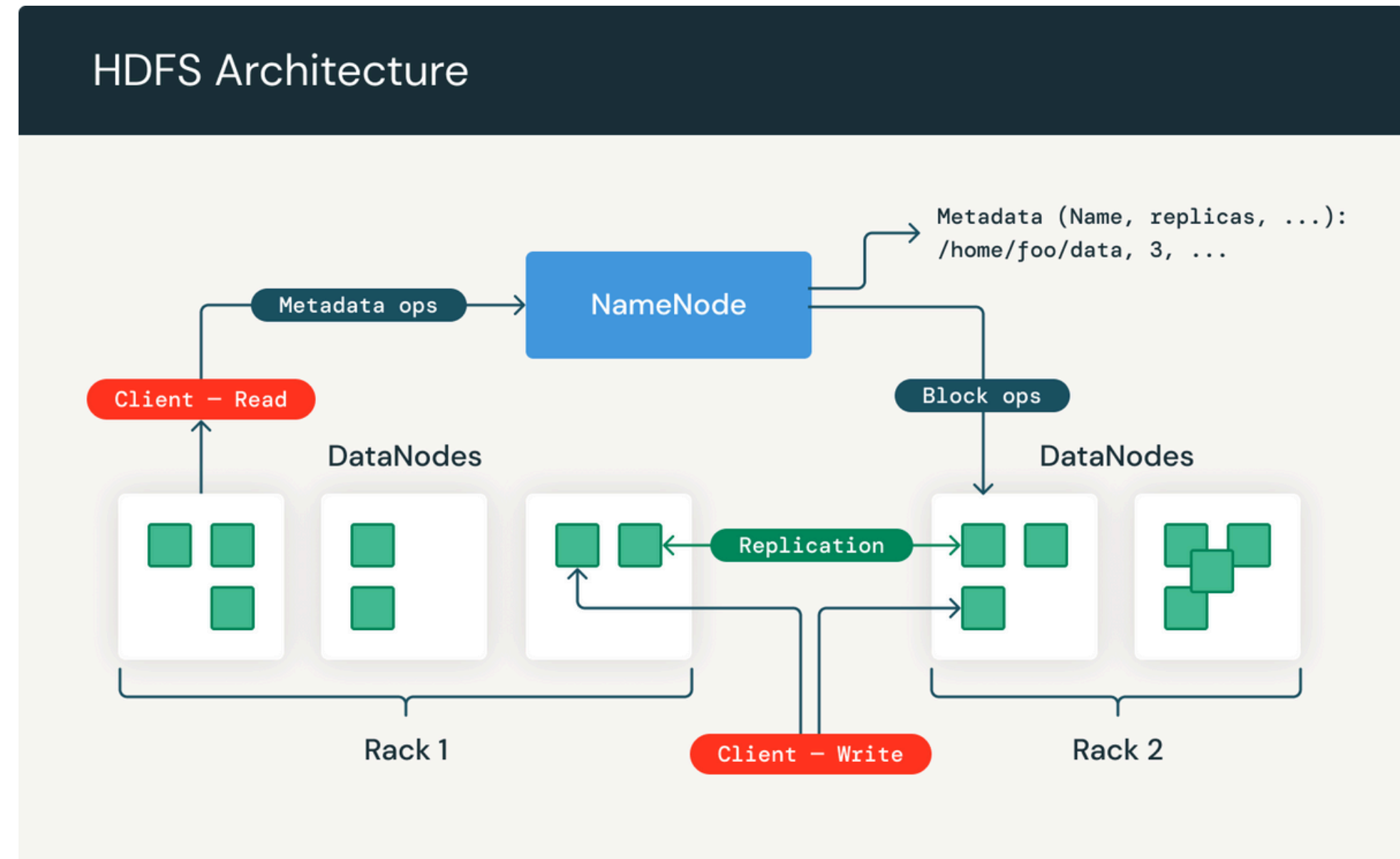
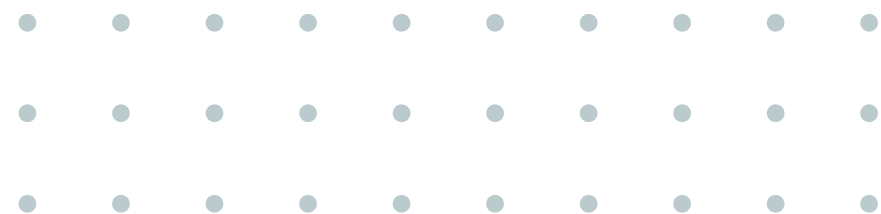
2. DataNode



Source: <https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs>

NameNode (Master Node)

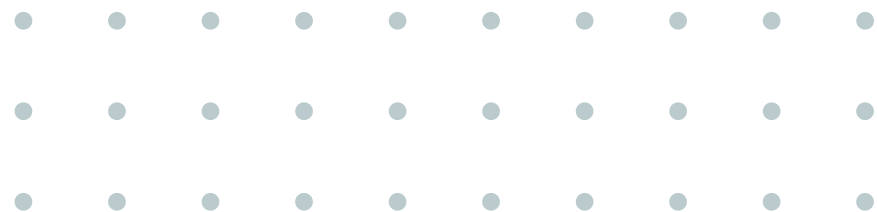
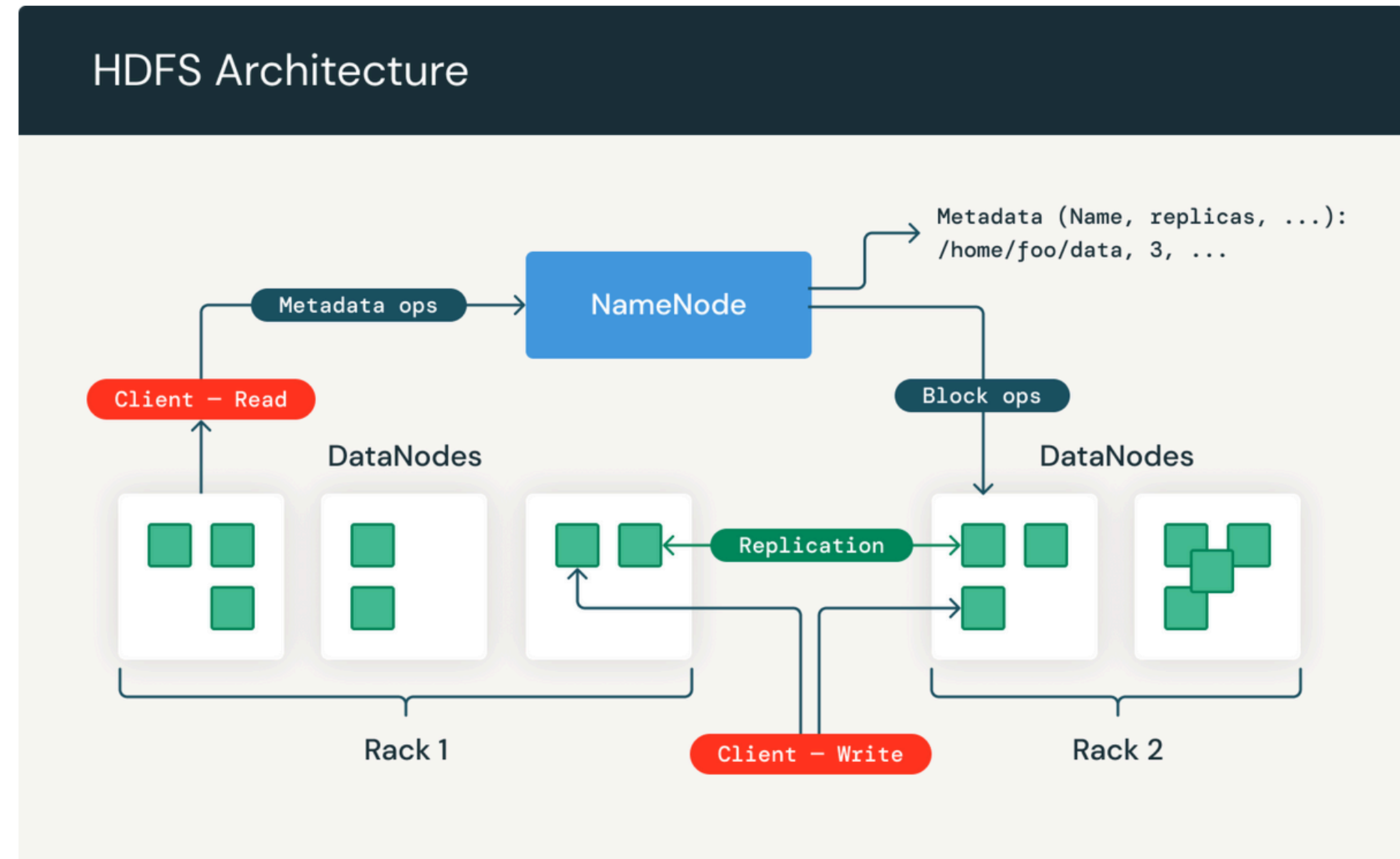
- Central **metadata** manager storing file structure and block locations.
- **Manage** DataNodes
 - Coordinates access to the filesystem but does not store data itself



Source: <https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs>

DataNode (Worker Node)

- Store actual **data blocks**
- Serving **read/write** requests
 - Perform tasks like block creation, deletion, and replication under NameNode's instructions.



Source: <https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs>

HDFS ARCHITECTURE

2 Core components

1.NameNode

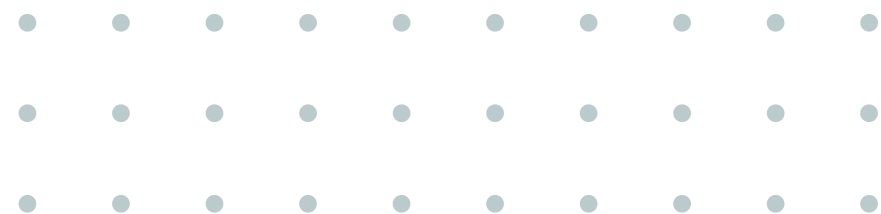
2.DataNode

3 additional components

1.Secondary NameNode

2.Job Tracker

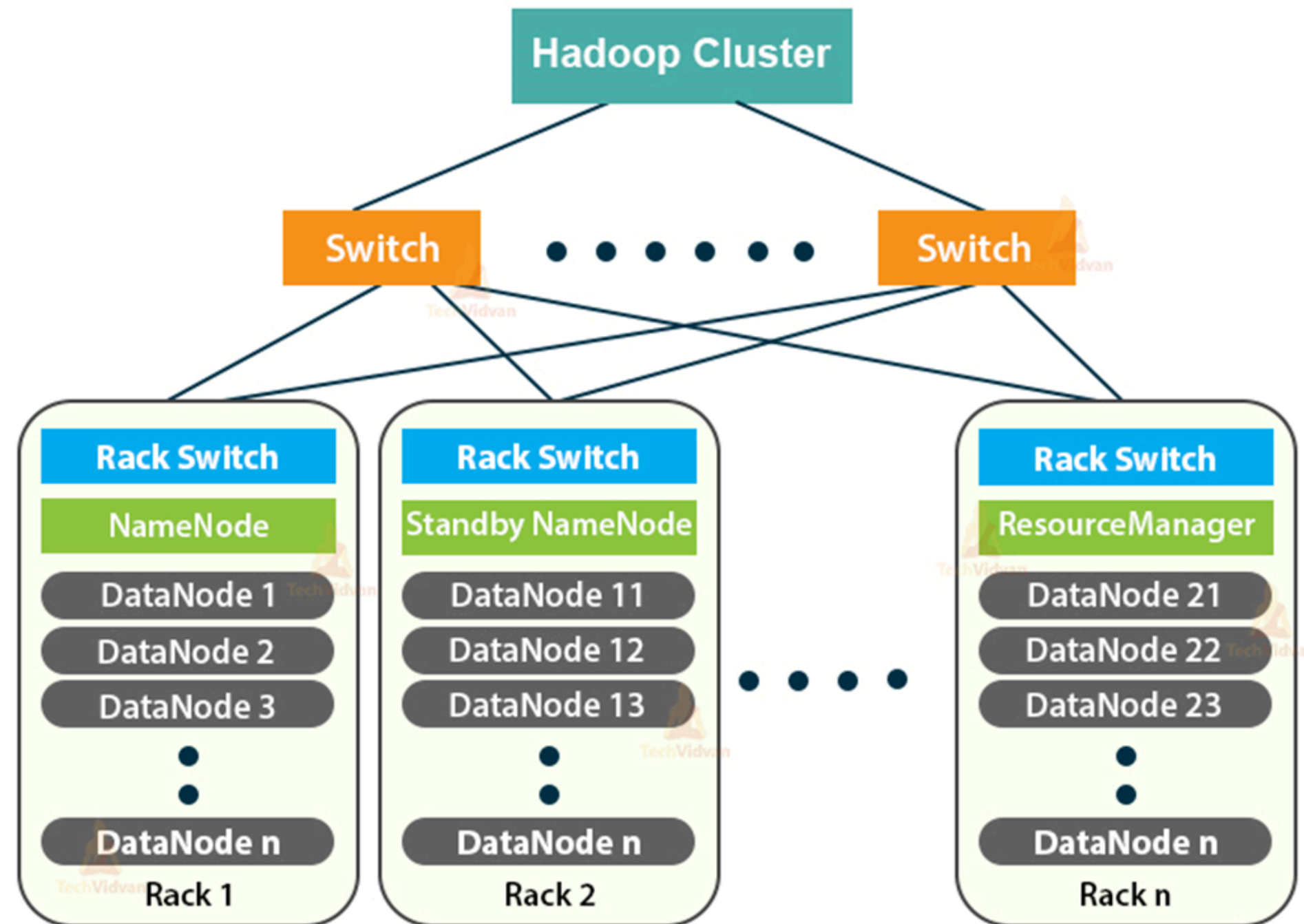
3.Task Tracker



Cluster



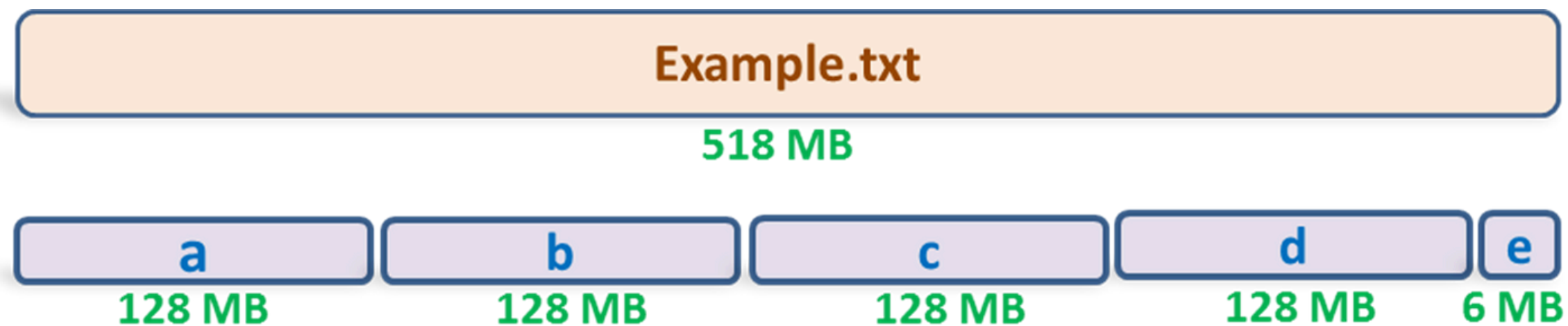
HDFS Cluster Architecture



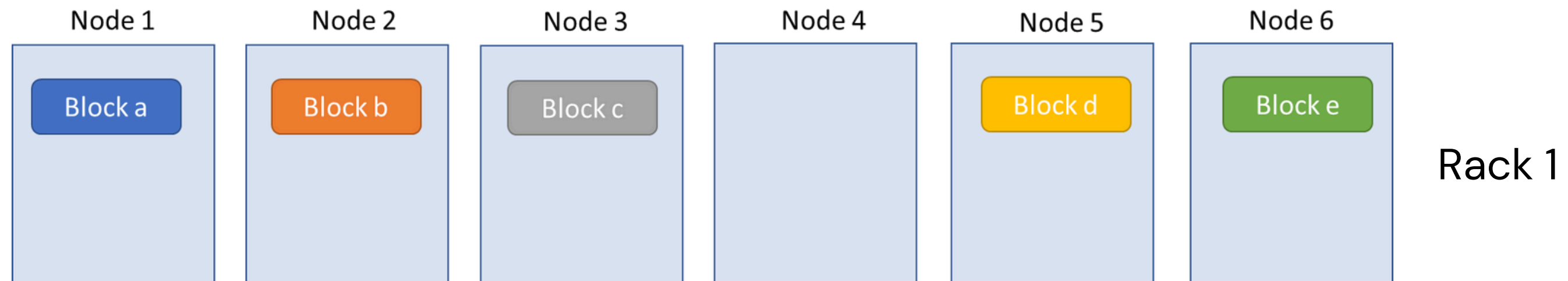
HDFS Data Blocks



- HDFS is a block-structured file system where large files are split into fixed-sized smaller chunks (default: **128 MB** or 256 MB)
- Each chunk is called a **block**
 - A smallest unit of data storage in HDFS



Data Distribution



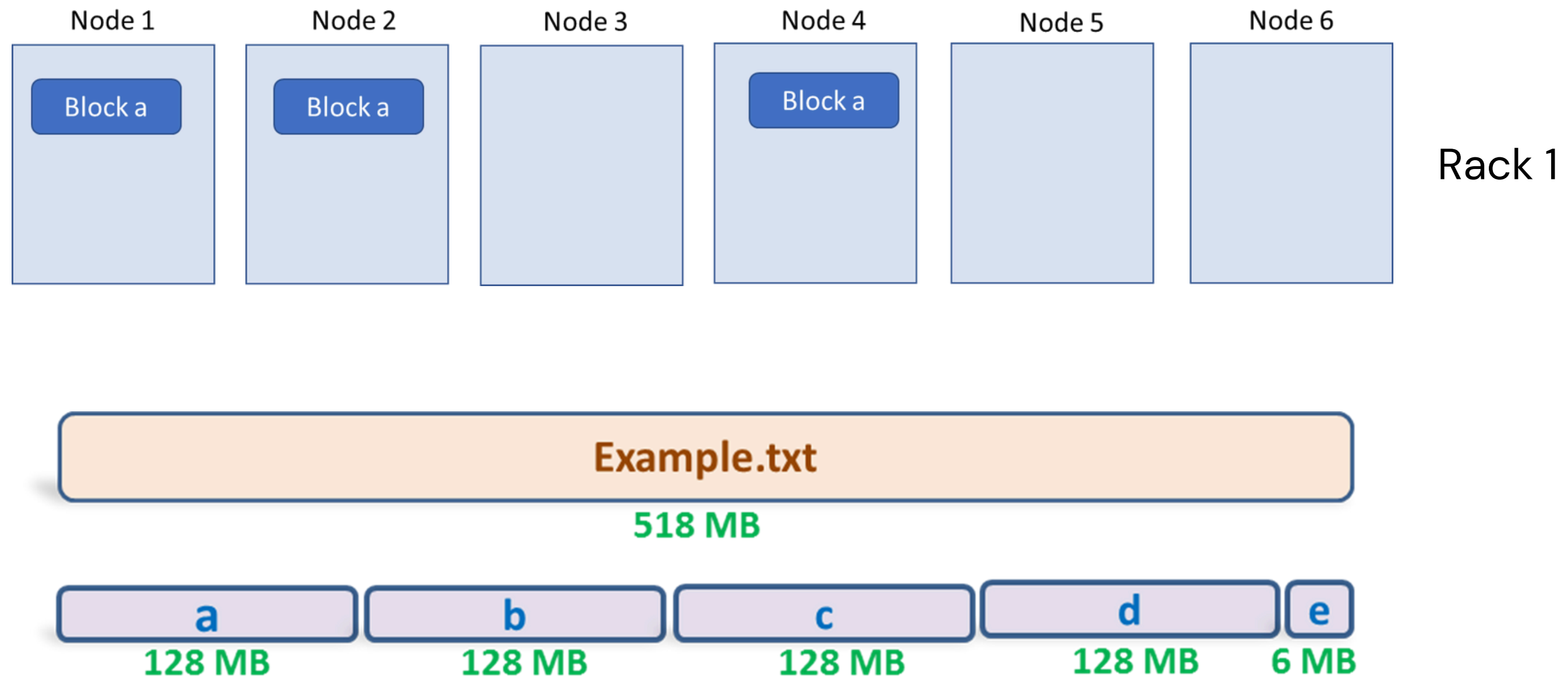
518 MB



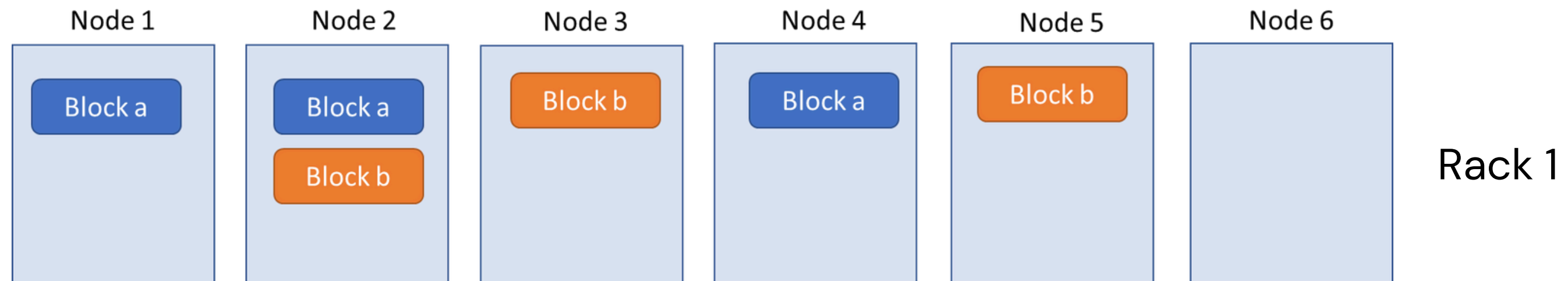
Data Replication



- Each block is replicated (default: 3 copies) across different DataNodes to ensure reliability (i.e. fault tolerance).



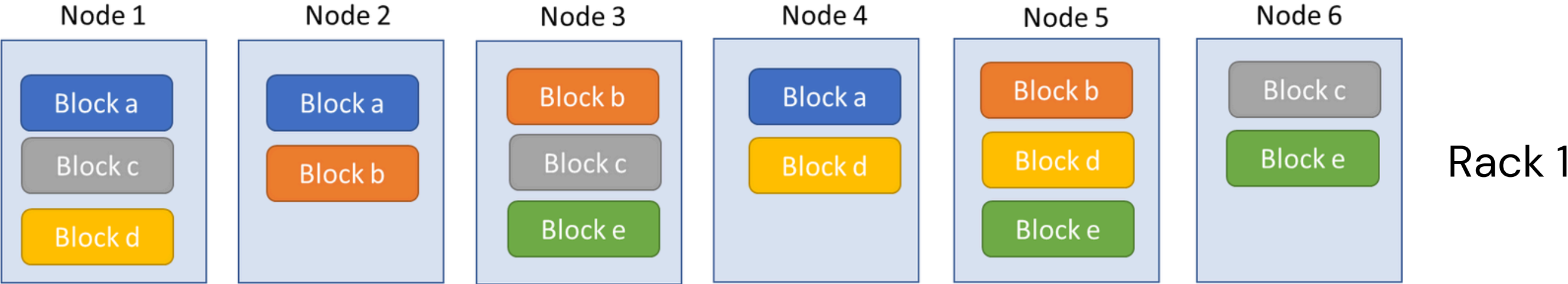
Data Replication



518 MB



Data Replication



Example.txt

518 MB



a

b

c

d

e

128 MB

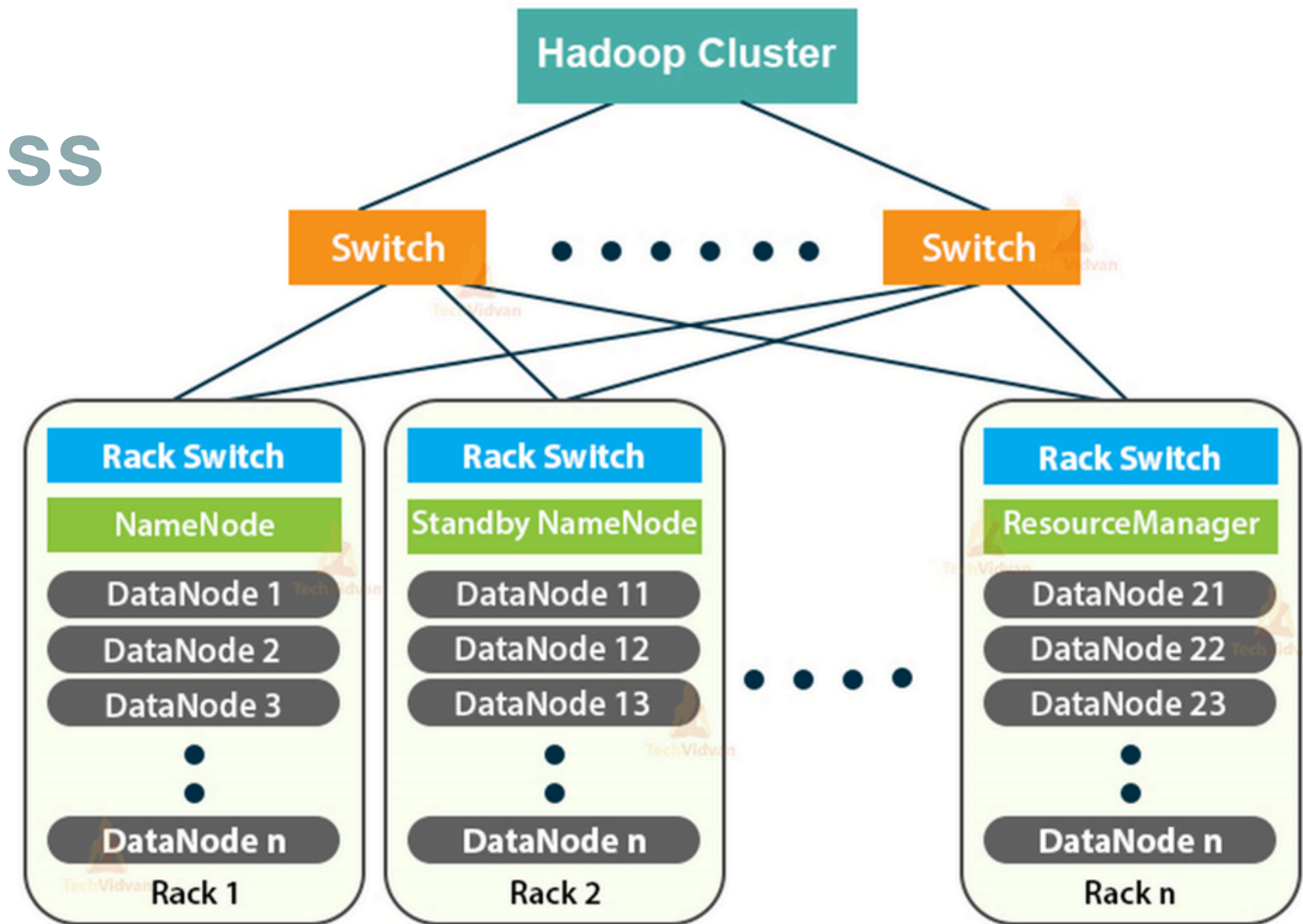
128 MB

128 MB

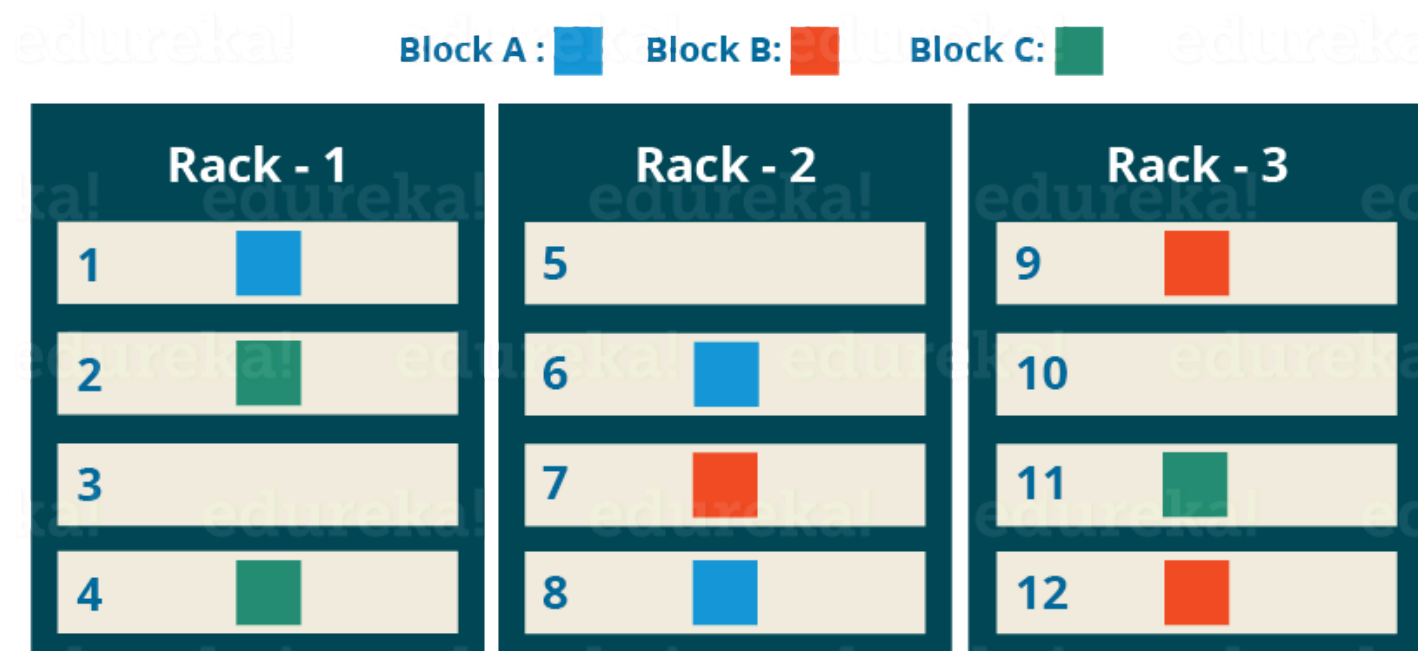
128 MB

6 MB

Rack Awareness

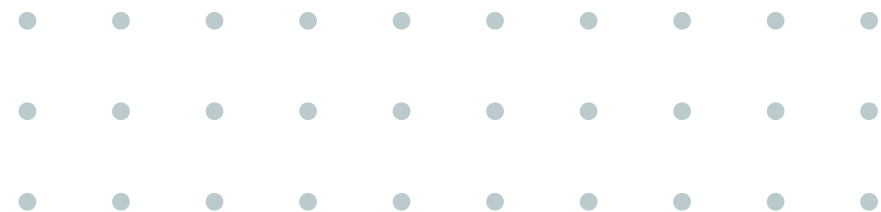
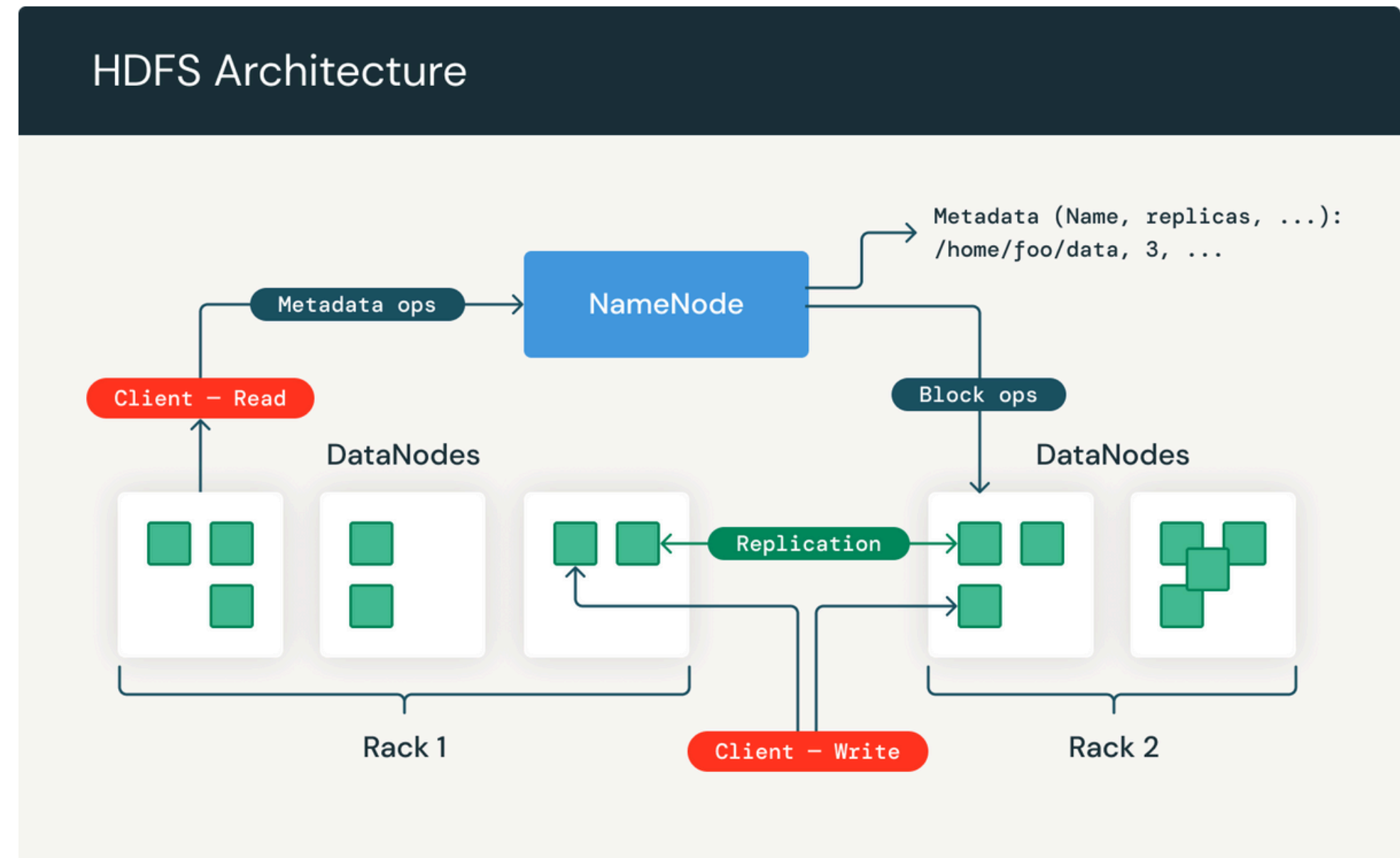


Rack Awareness Algorithm



HDFS READ/WRITE ARCHITECTURE

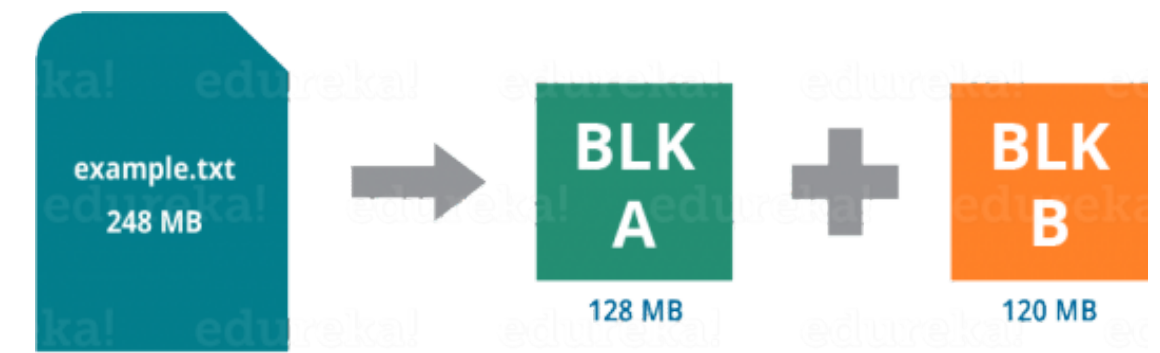
“Write Once – Read Many”



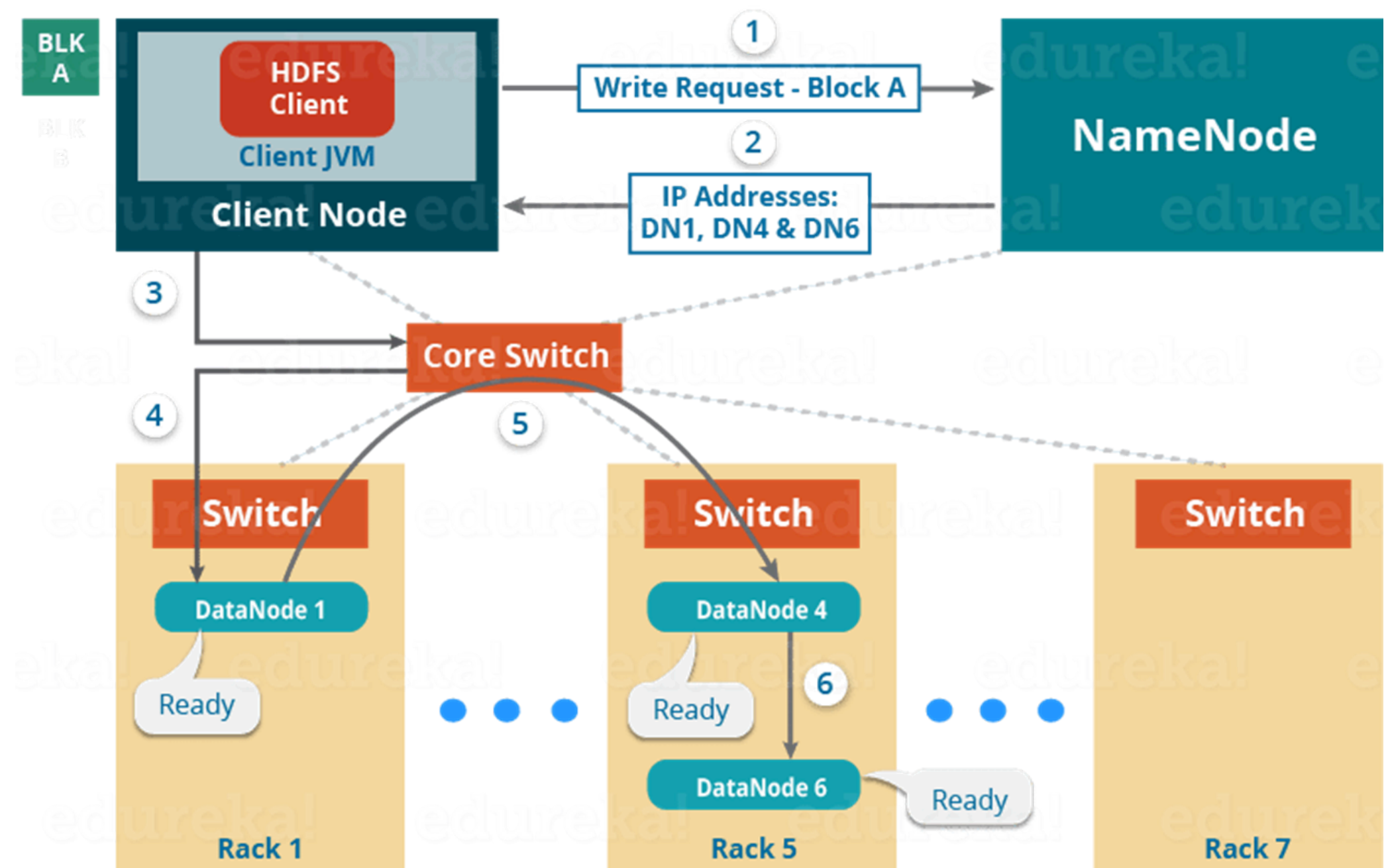
Source: <https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs>

Write Operation

1. Pipeline Setup

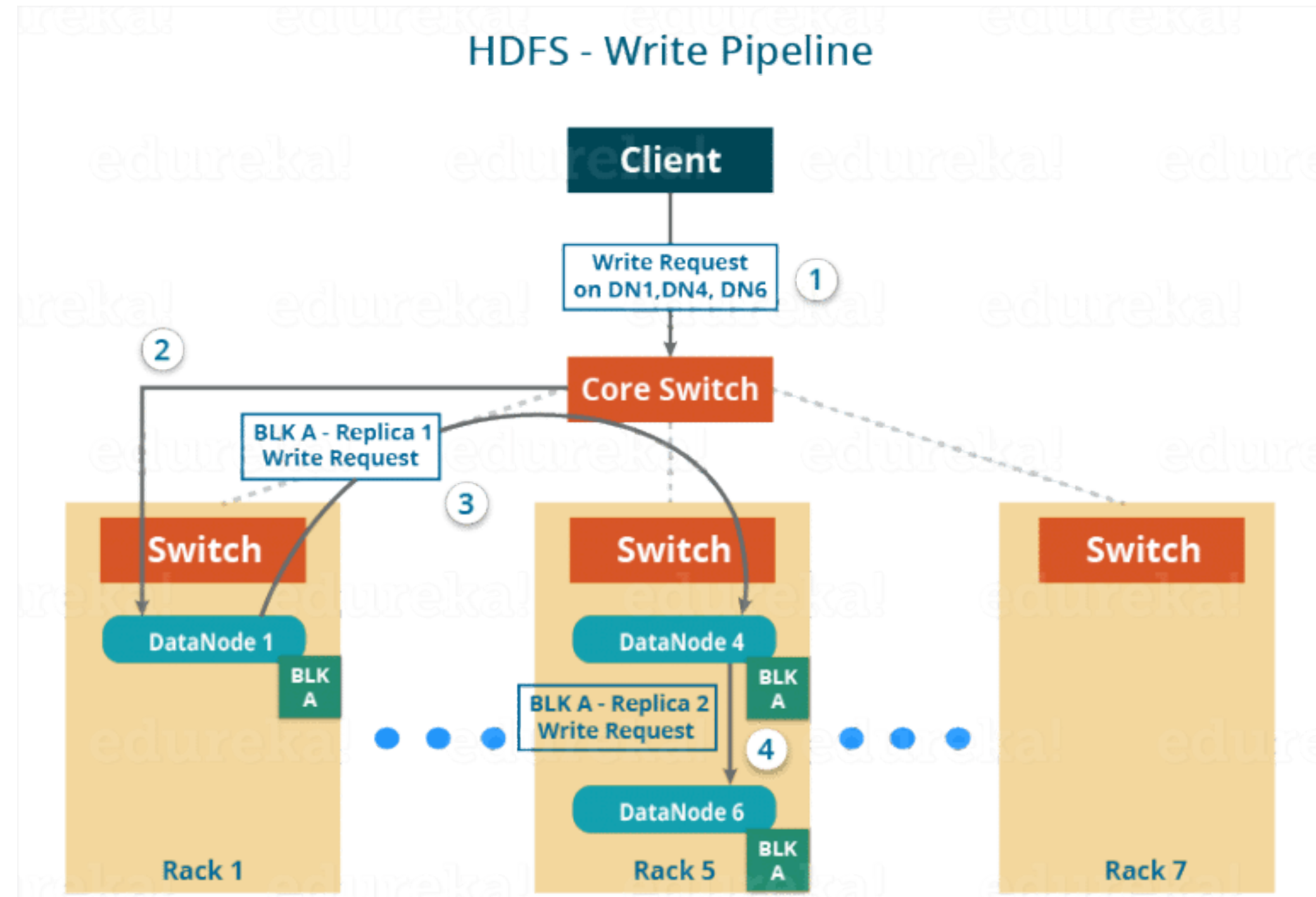
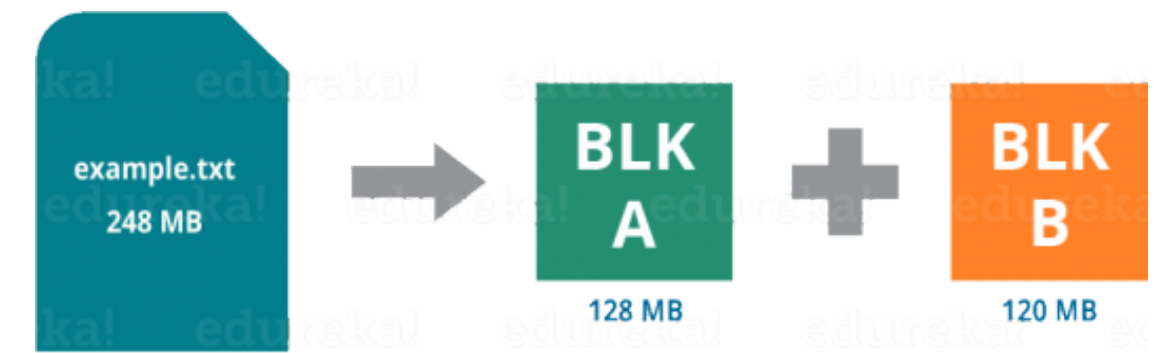


Setting up HDFS - Write Pipeline



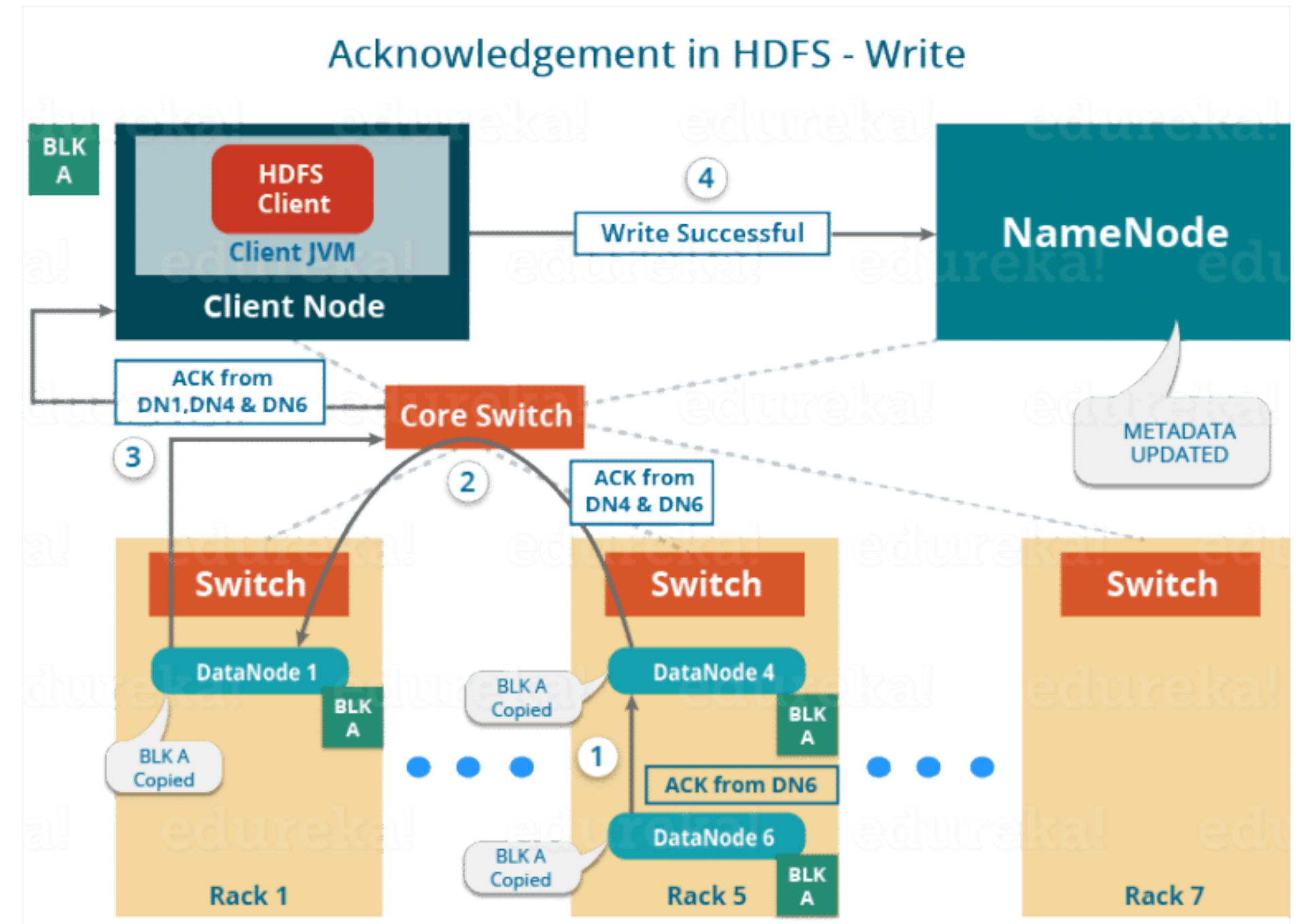
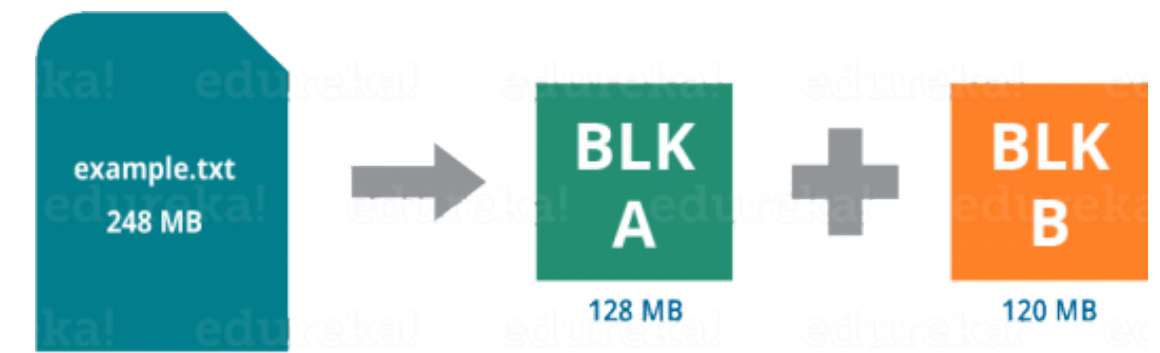
Write Operation

1. Pipeline Setup
2. Data Streaming



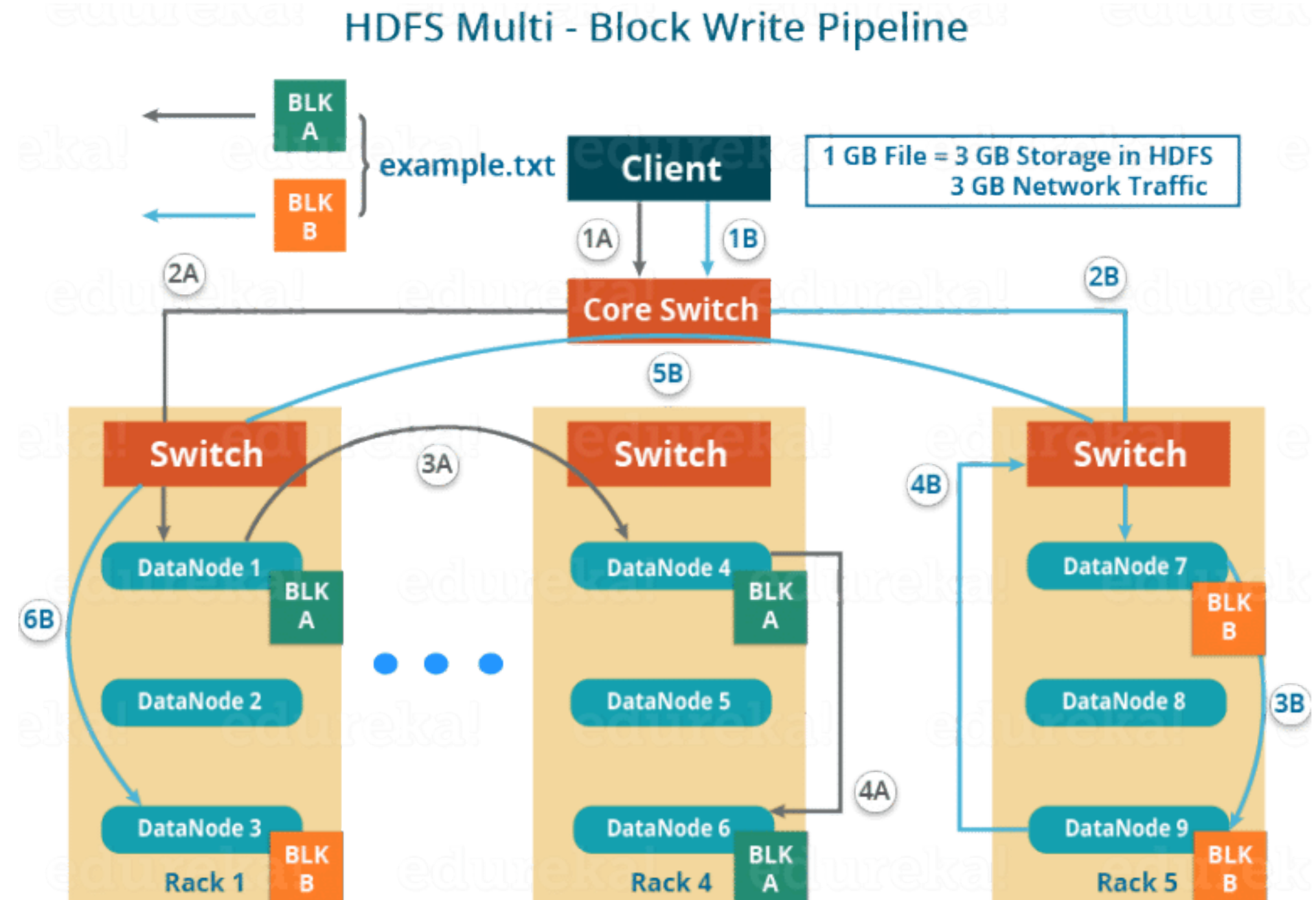
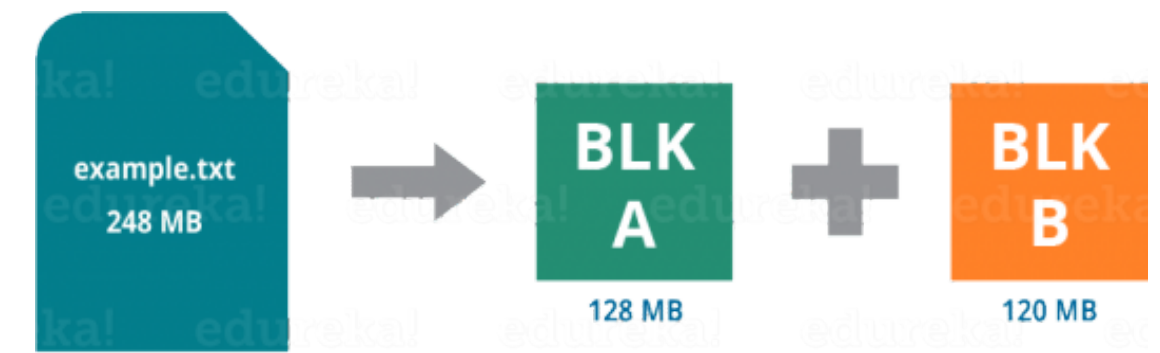
Write Operation

1. Pipeline Setup
2. Data Streaming
3. Acknowledgement

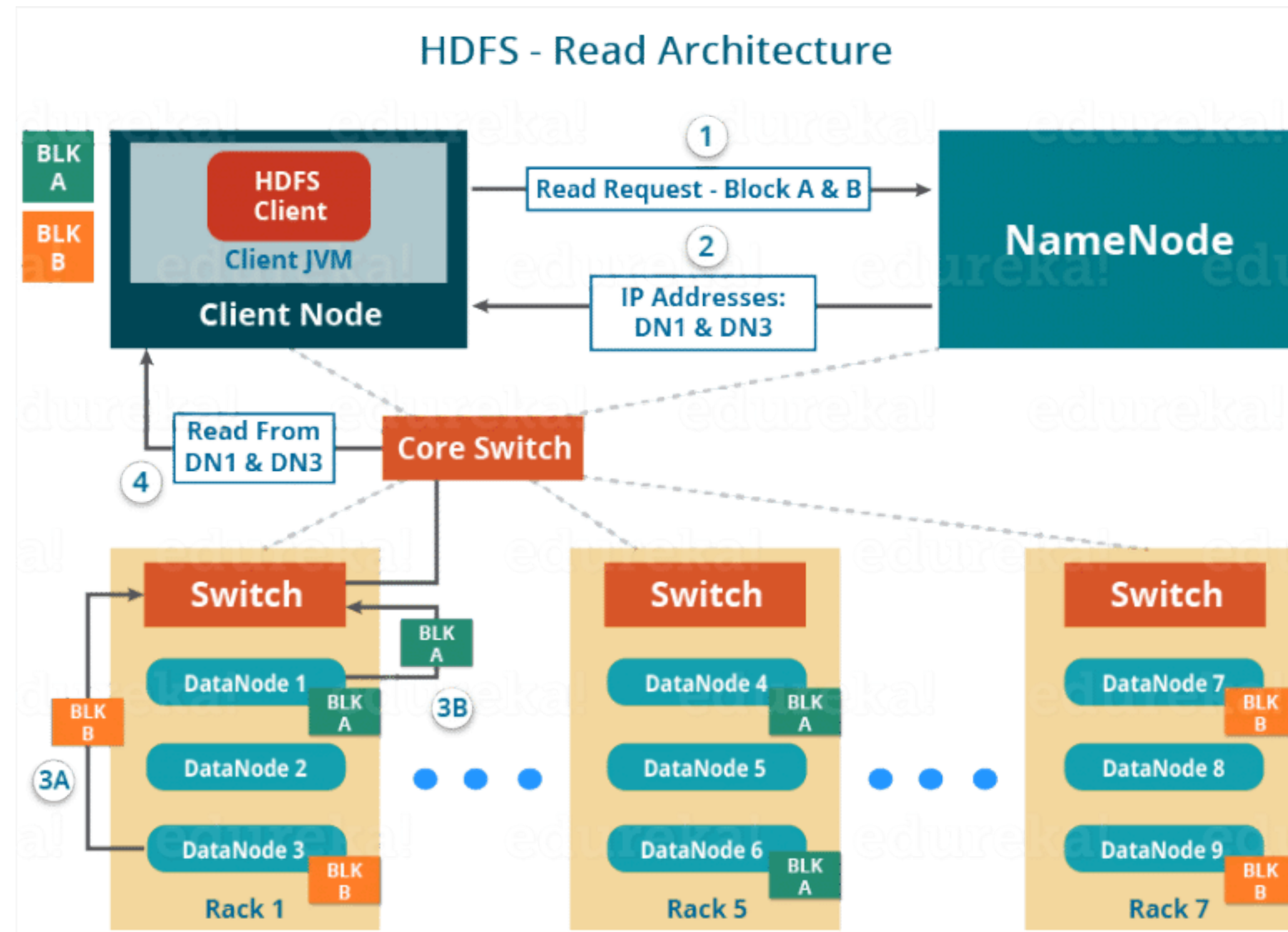
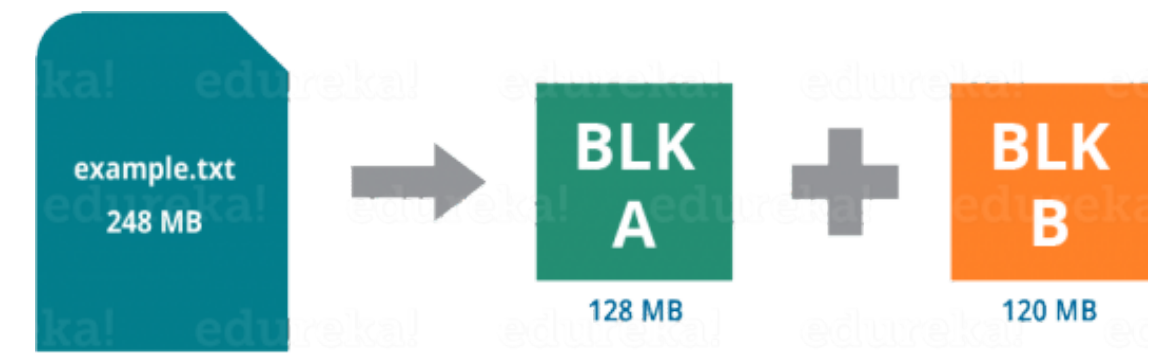


Write Operation

1. Pipeline Setup
2. Data Streaming
3. Acknowledgement



Read Operation



02.

MAPREDUCE

06026213 BIG DATA SYSTEMS

What is MapReduce?

- A programming model for processing large datasets in a **distributed and parallel** manner
- Divides a task into smaller sub-tasks and processes them independently across multiple nodes in a cluster.

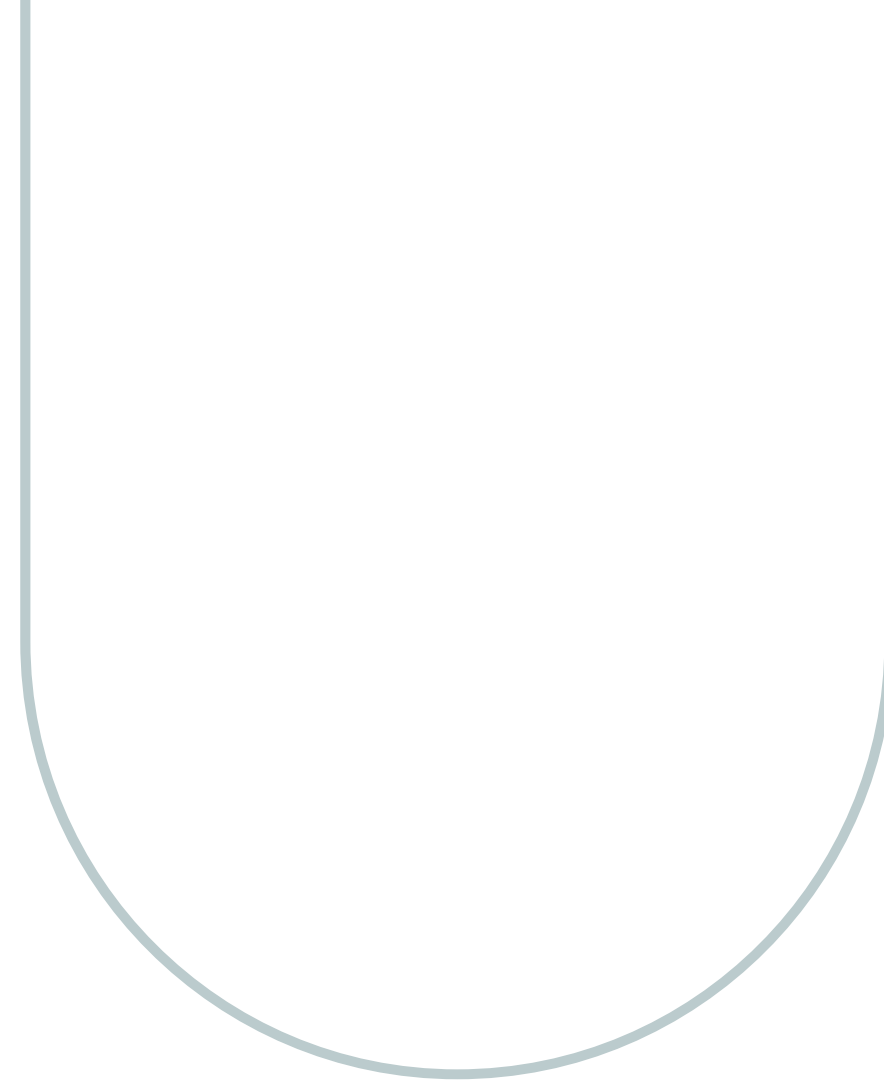


Origin of MapReduce

- First introduced by Google in a 2004 research paper titled "**MapReduce: Simplified Data Processing on Large Clusters**" by Jeffrey Dean and Sanjay Ghemawat.
- Aimed to address the challenges of processing and generating massive datasets at Google's scale.



Origin of MapReduce



Problem Statement:

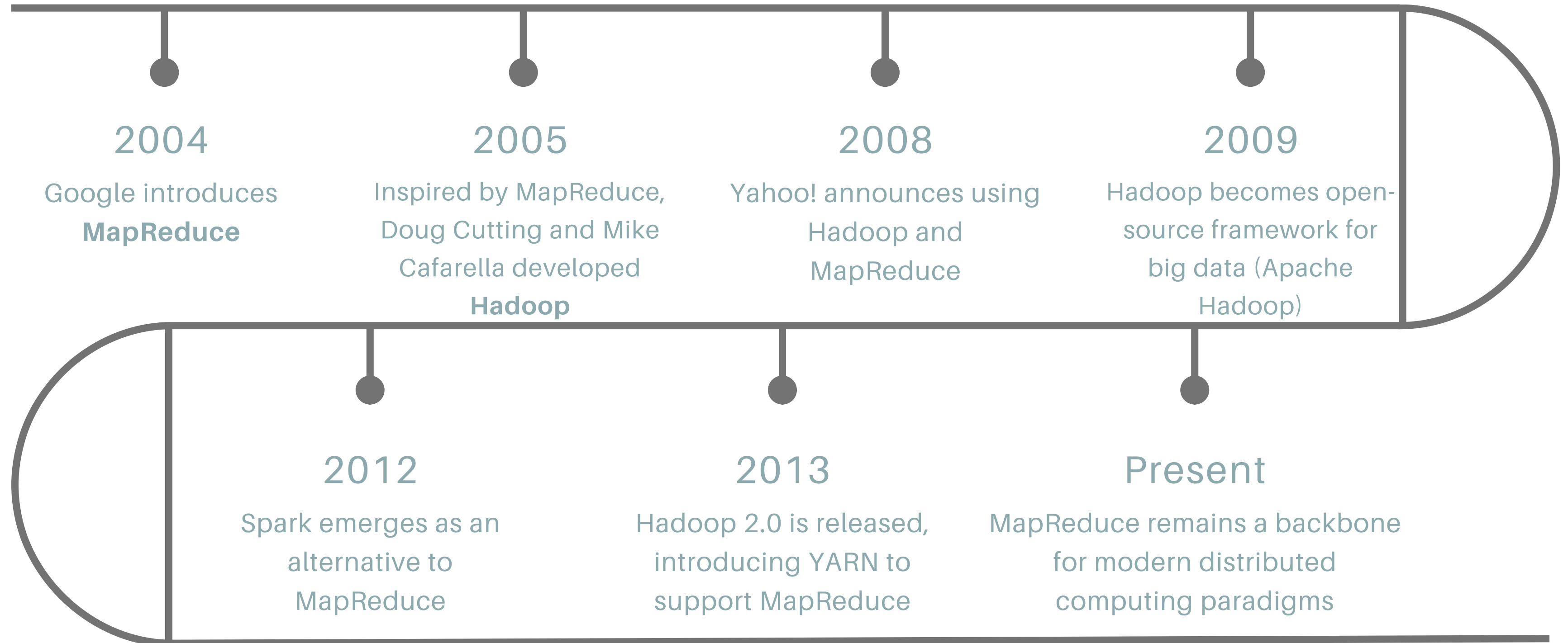
- Efficiently process web-scale data (e.g., billions of web pages, logs, and user data) using a distributed infrastructure.
- Simplify parallel computing by abstracting the complexities of task distribution, fault tolerance, and system management.

Origin of MapReduce

Inspiration from Functional Programming Concepts:

- Map: Applies a function to a list of elements to produce a set of results.
- Reduce: Aggregates these results into a final output.

Timeline of MapReduce



Processing Analogy



Aj. Bank



Student A



Student B

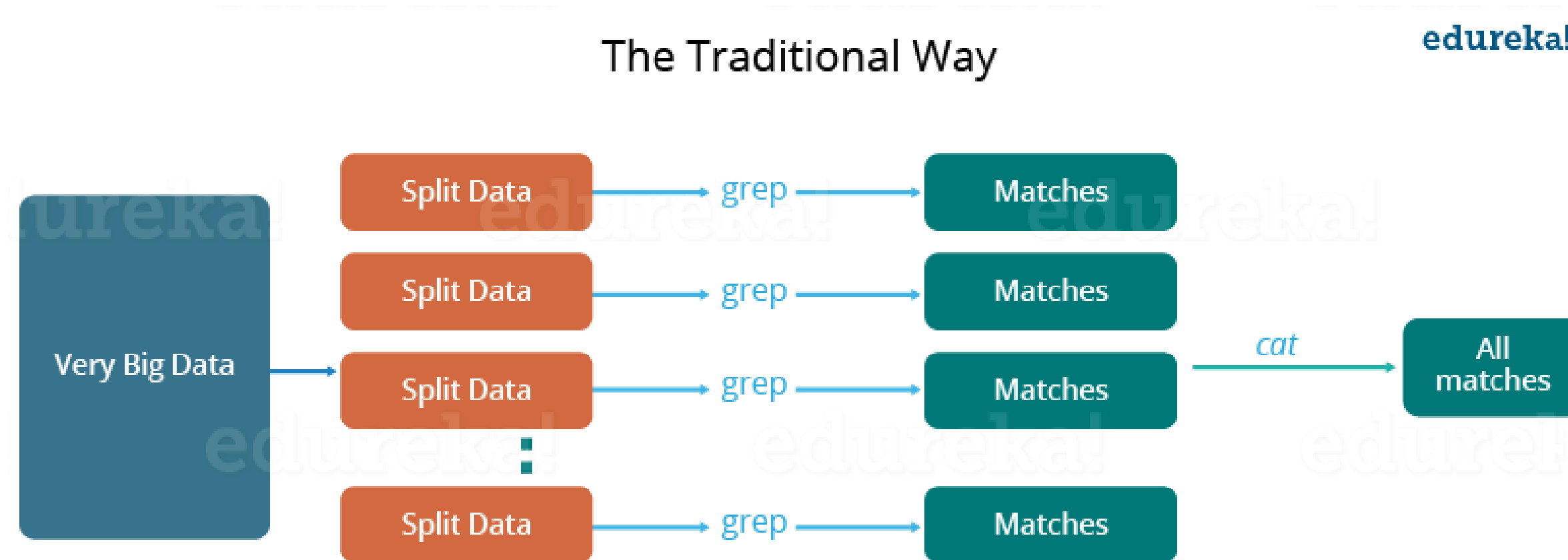


Student C



Student D

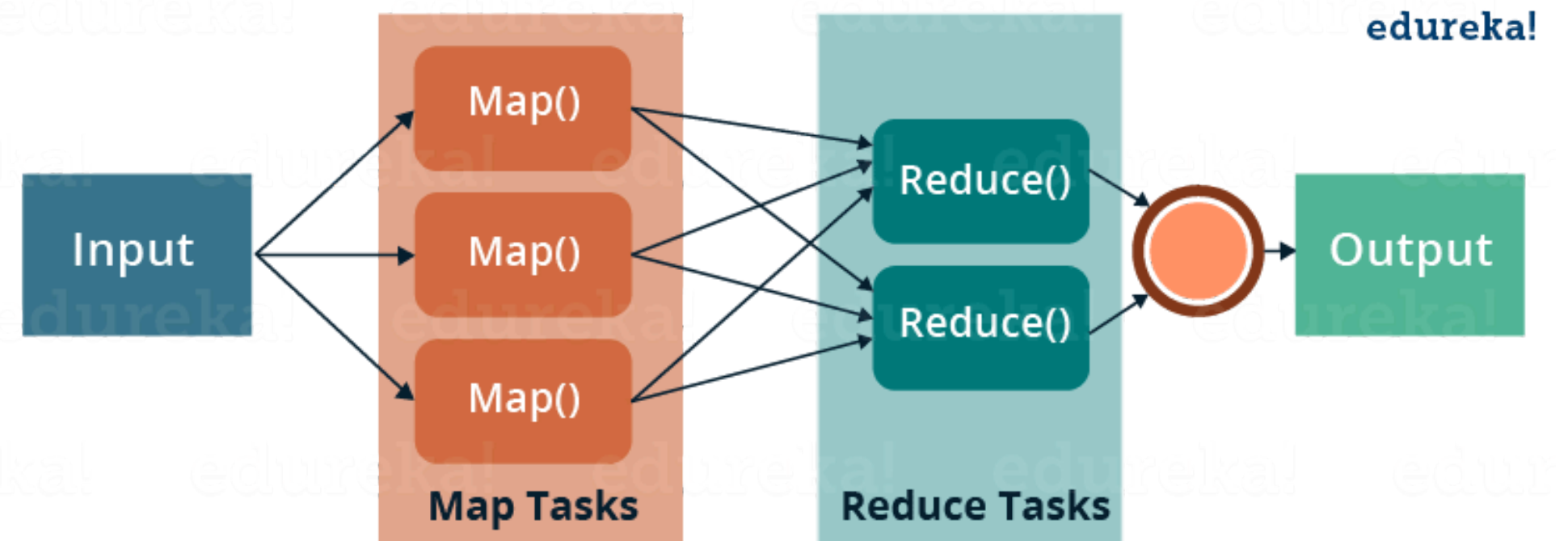
Parallel and Distributed Processing: Traditional



MapReduce

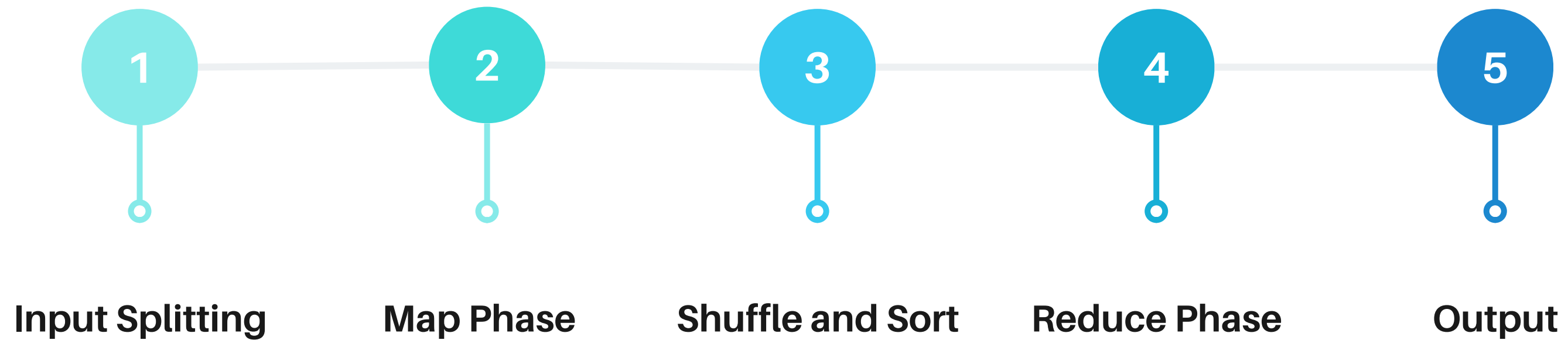
Two main tasks

- Map
 - a block of data is read and processed to produce key-value pairs as intermediate outputs.
- Reduce
 - Receives the key-value pair from multiple map jobs and aggregates into a final output.



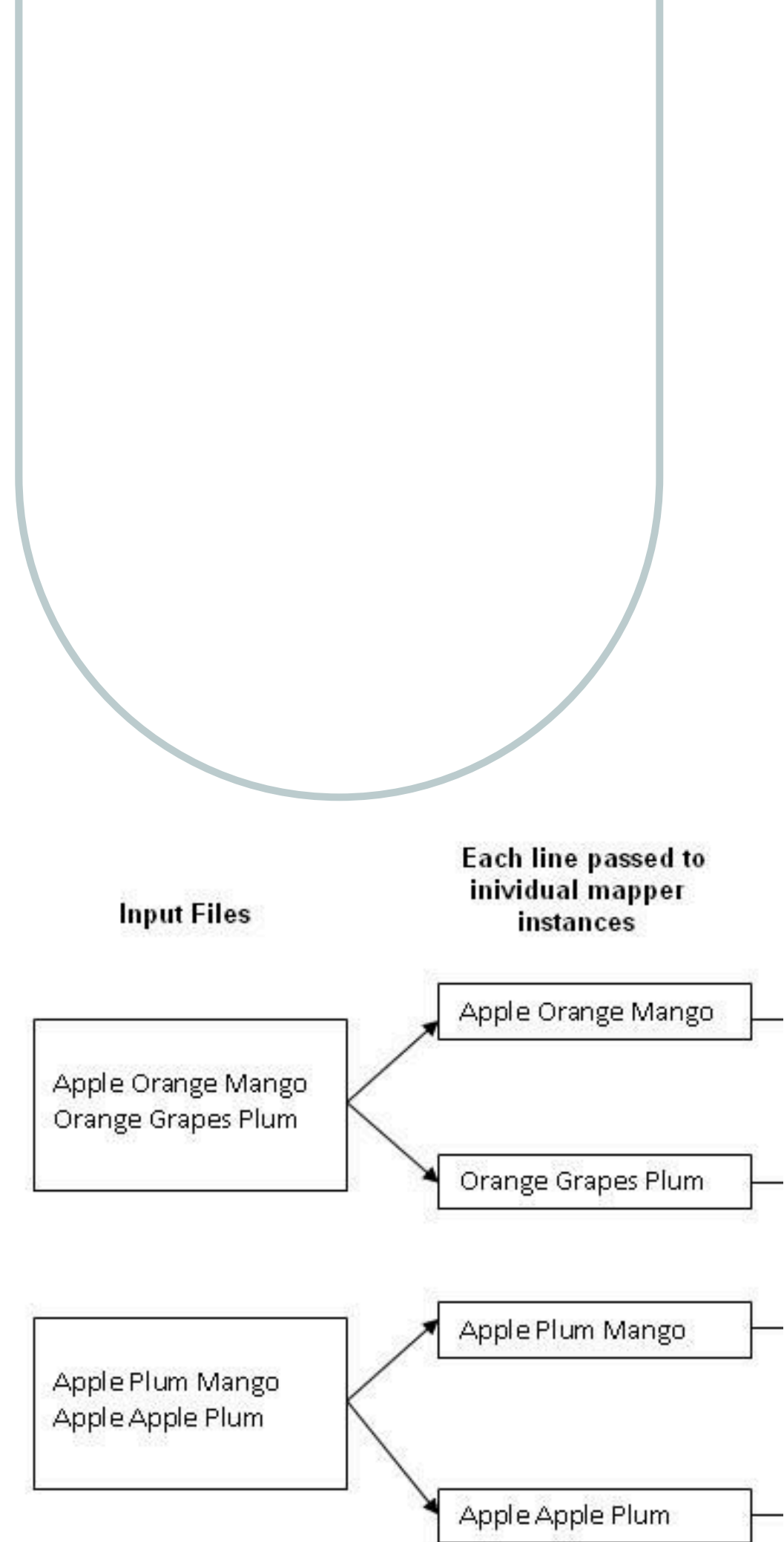
Source: <https://www.edureka.co/blog/mapreduce-tutorial/>

Key Steps in the MapReduce Workflow



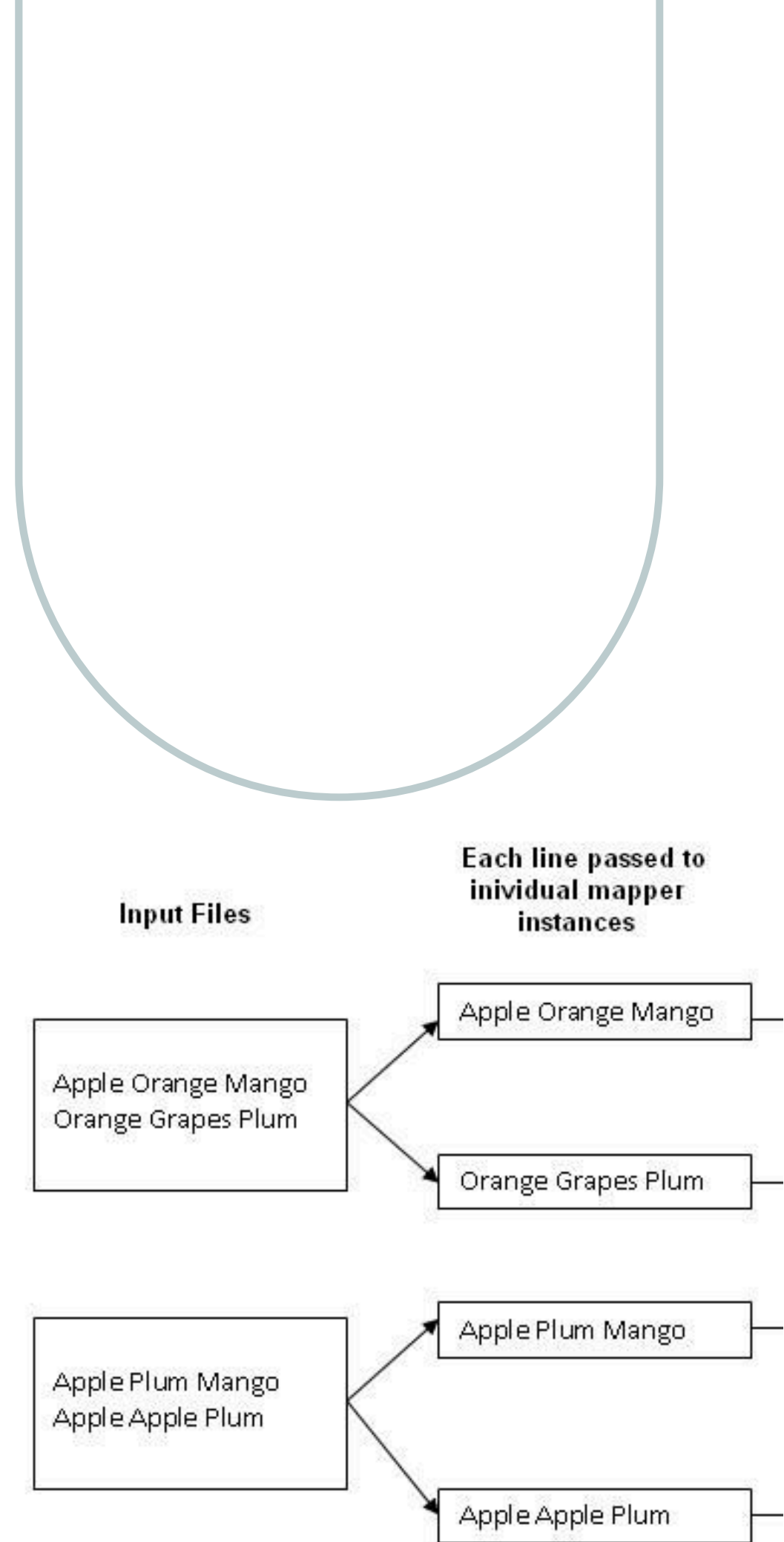
1. Input Splitting

- The input dataset is divided into smaller chunks (splits), stored across the cluster nodes in HDFS.
- Each split corresponds to a block of data in HDFS, typically 128 MB in size.



Overlapping Challenge

- When splitting large datasets into blocks for processing, a single word or record might span across two blocks.
- Example:
 - Block 1 ends with the text: "This is a sample sen-"
 - Block 2 starts with: "tence with some overlap"

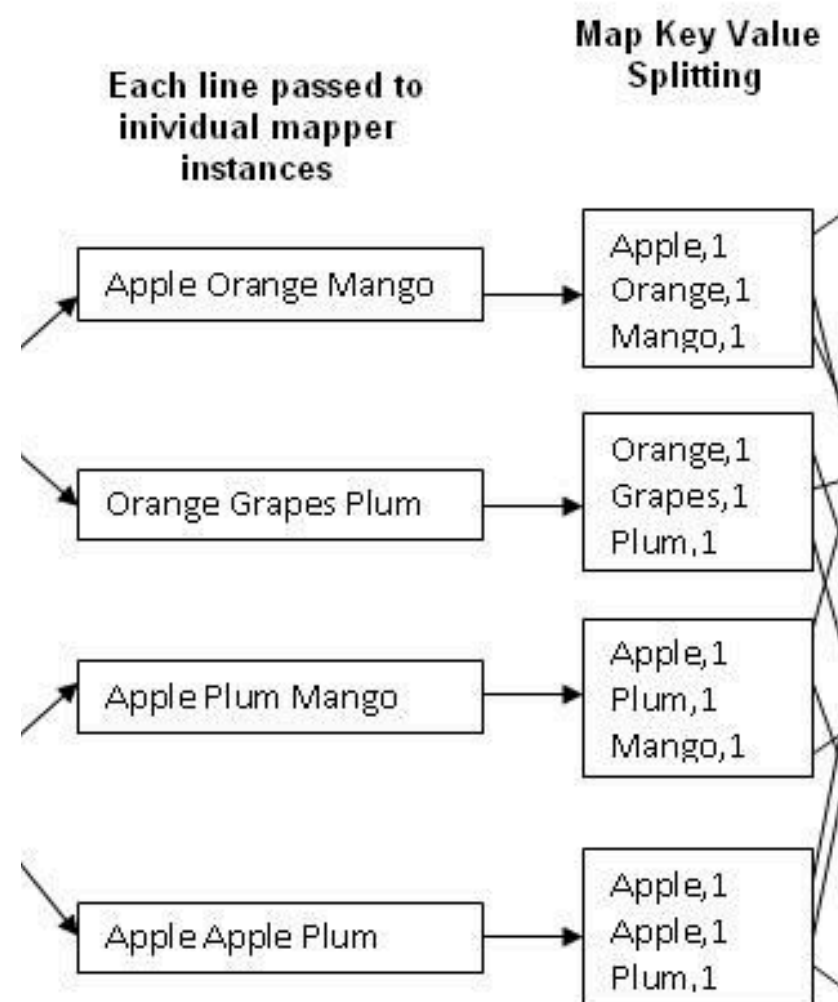


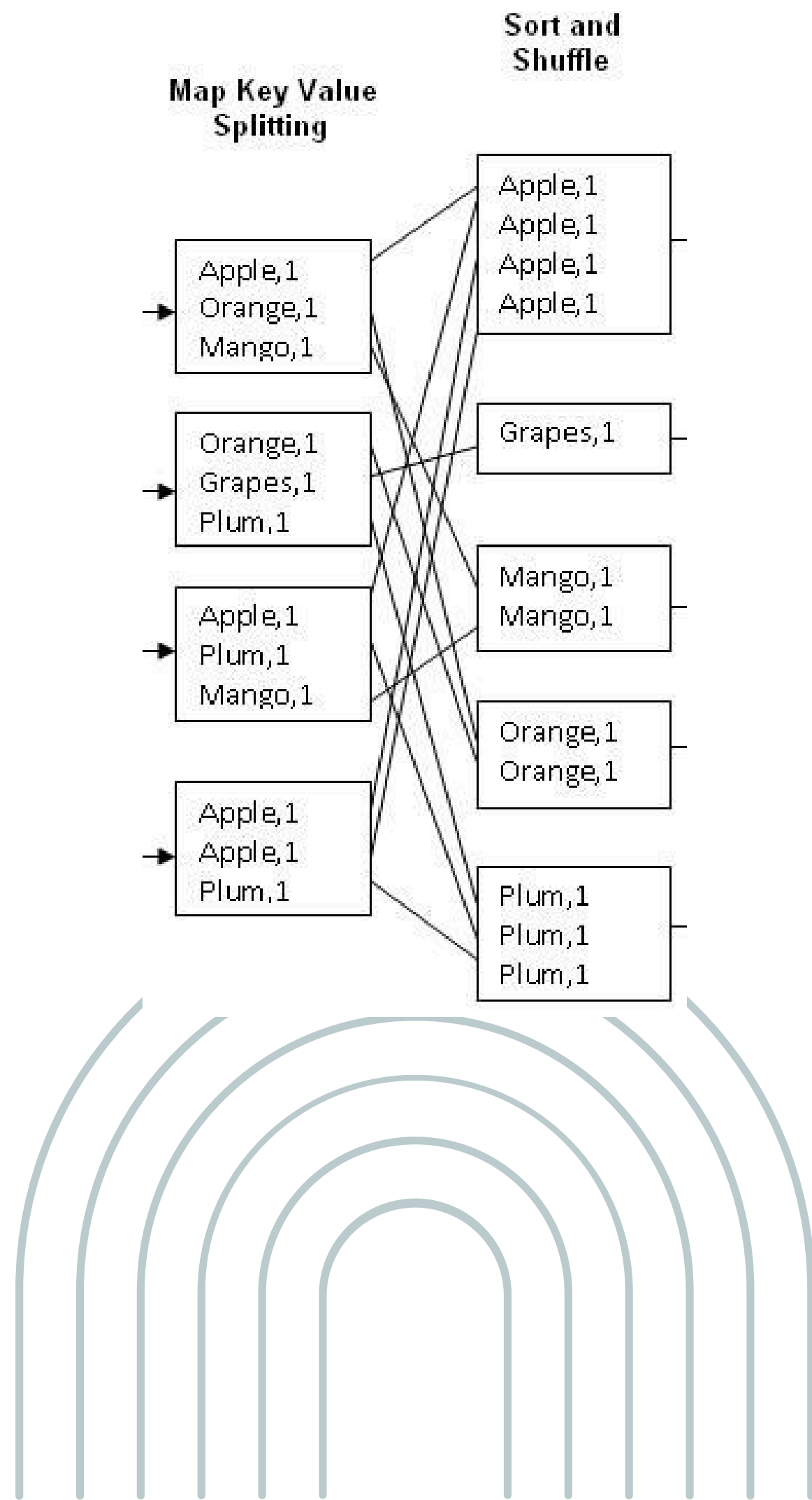
Cross-Node Splitting Challenge

- Some input records may span multiple blocks stored on different nodes.
- Example:
 - Block 1 ends on Node A
 - Block 2 begins on Node B

2. Map Phase

- Mapper functions operate on each input split to generate intermediate key-value pairs.
- Input records are processed line-by-line or item-by-item.
- Each record is transformed into one or more key-value pairs.



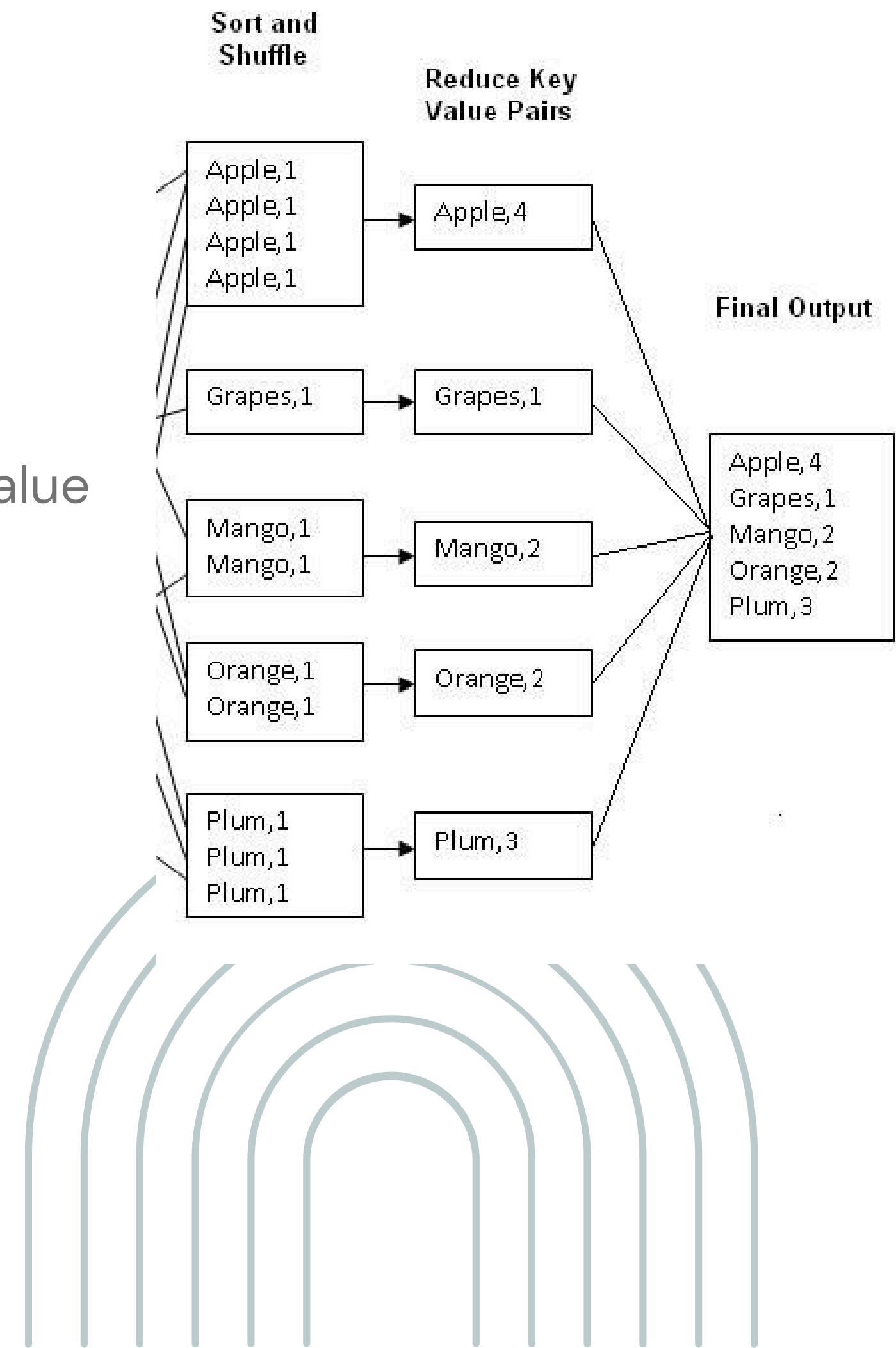


3. Shuffle and Sort

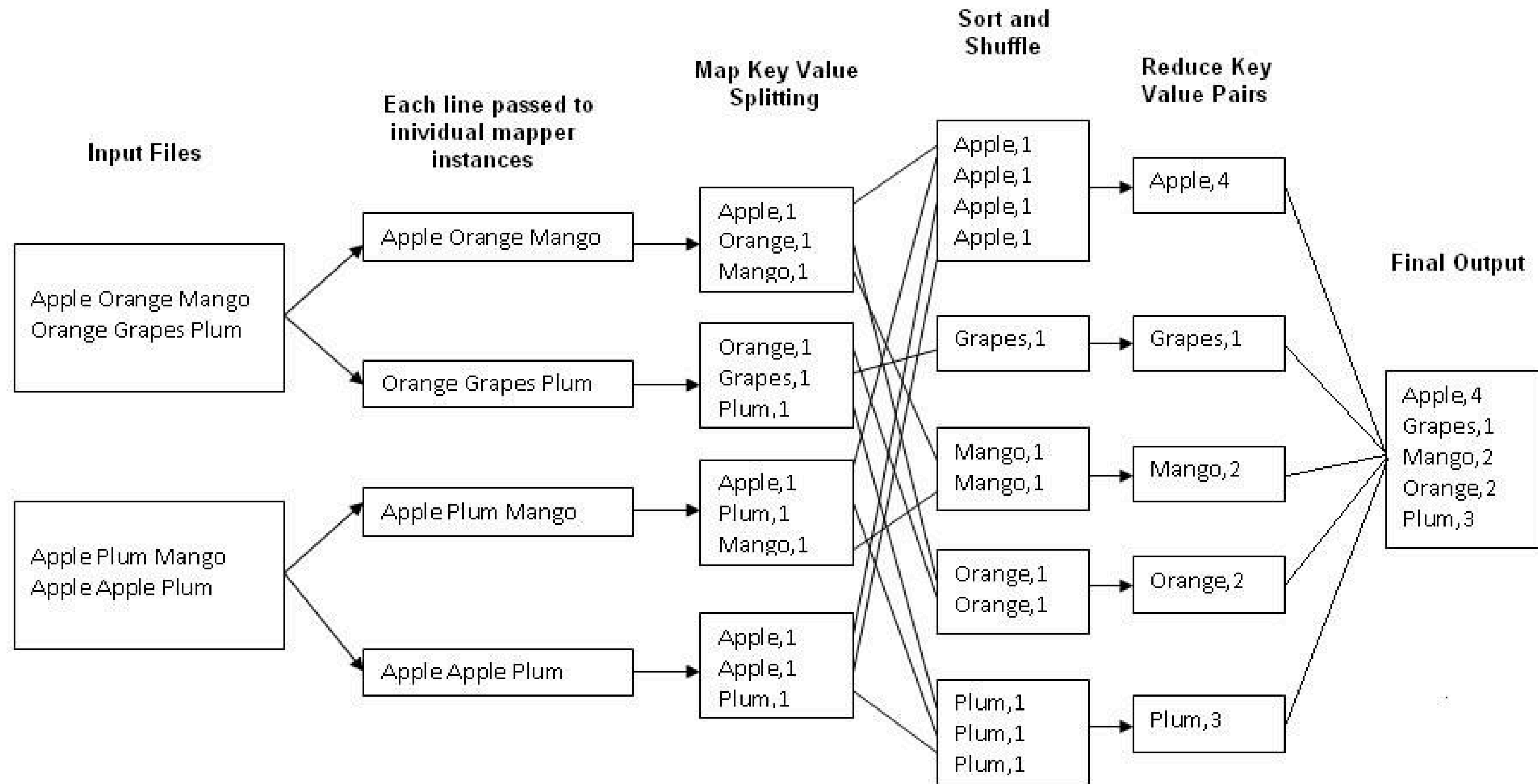
- The intermediate key-value pairs are grouped and sorted by key.
- Data is redistributed across the cluster so that all pairs with the same key are sent to the same reducer.
- Sorting ensures efficient grouping of values by key.

4. Reduce Phase

- Aggregate or process the grouped key-value pairs to generate the final output.
- The final output is written back to HDFS, distributed across the cluster.

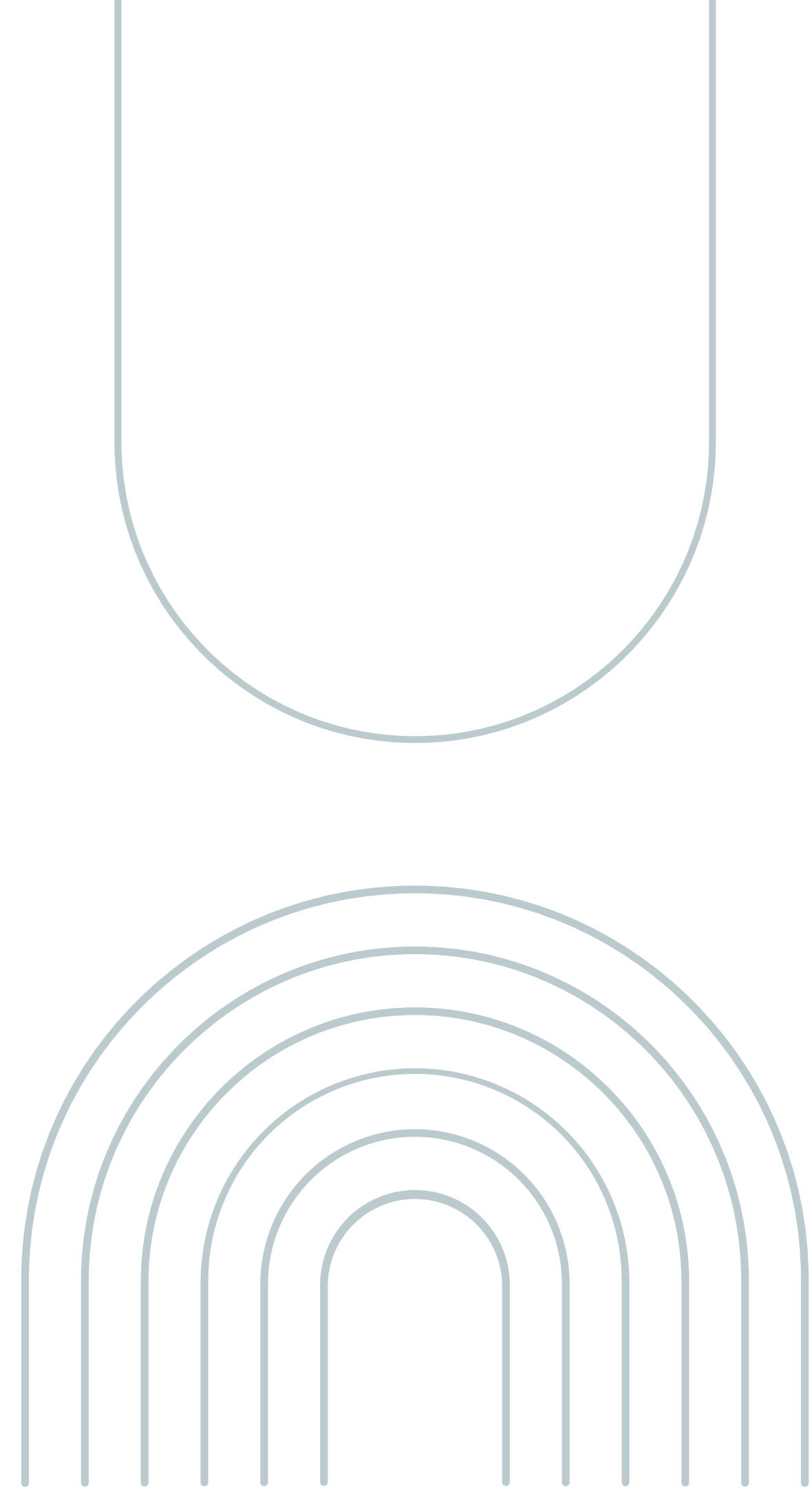


Word Count Example

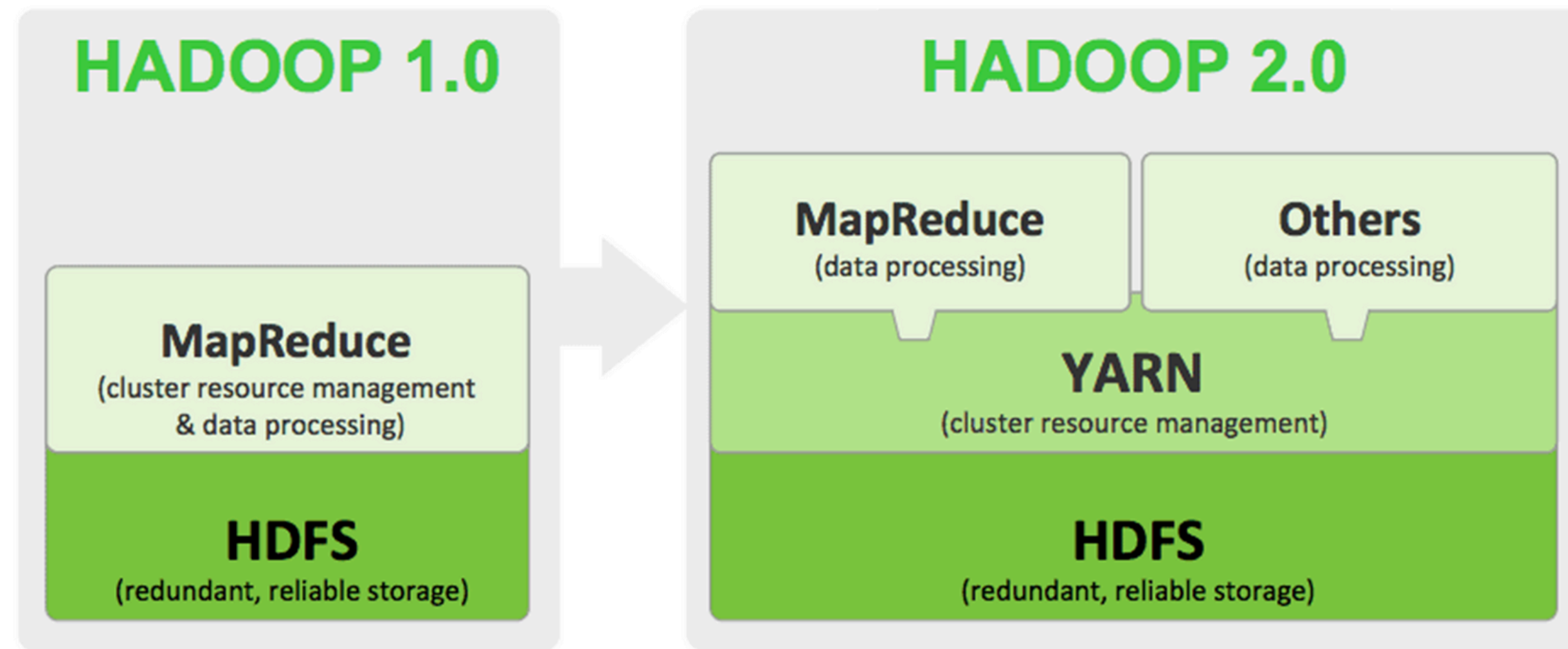


Key Features of MapReduce Process

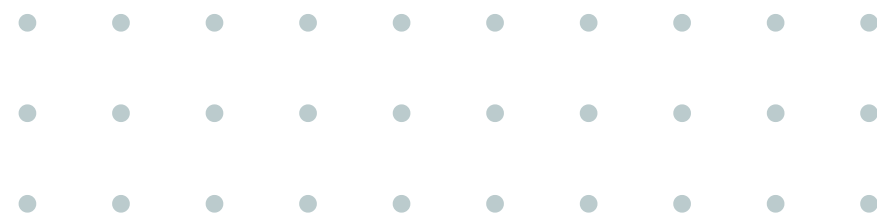
- Parallel Processing: Divide and Conquer Paradigm
- Fault Tolerance: Automatically reassigns tasks in case of node failures.
- Data Locality: Processes data on nodes where it is stored.
- Simplified Workflow: Developers focus on writing Map and Reduce functions.



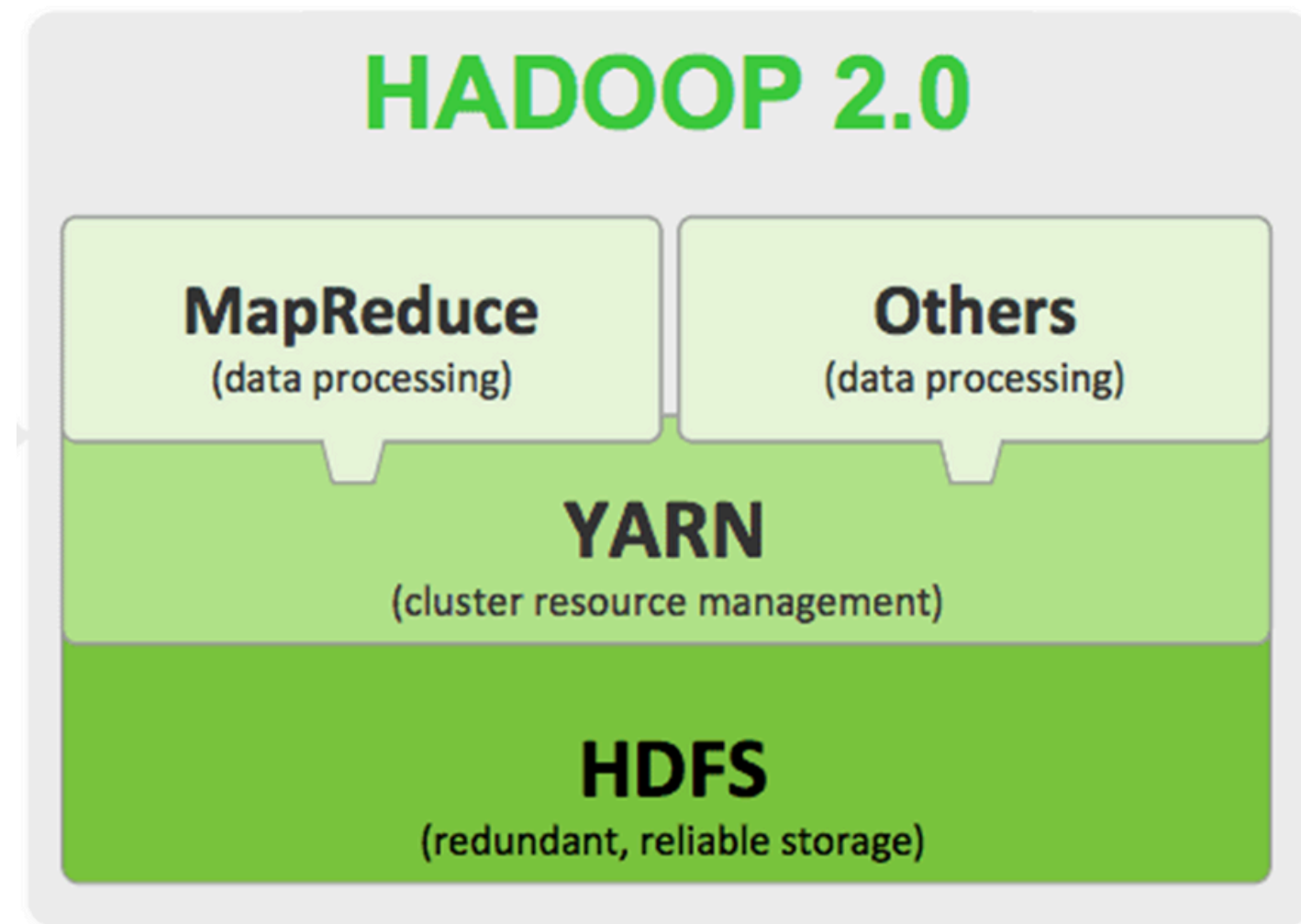
How Hadoop manages MapReduce



Source: <https://dataconomy.com/2014/06/hadoop-components-need-know/>

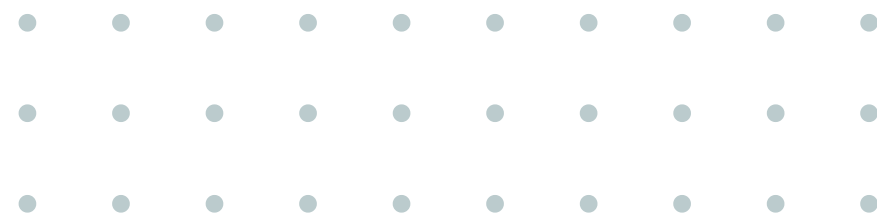


Hadoop 2 vs Hadoop 3



- Multiple NameNodes
- Able to runs on container
- Etc.

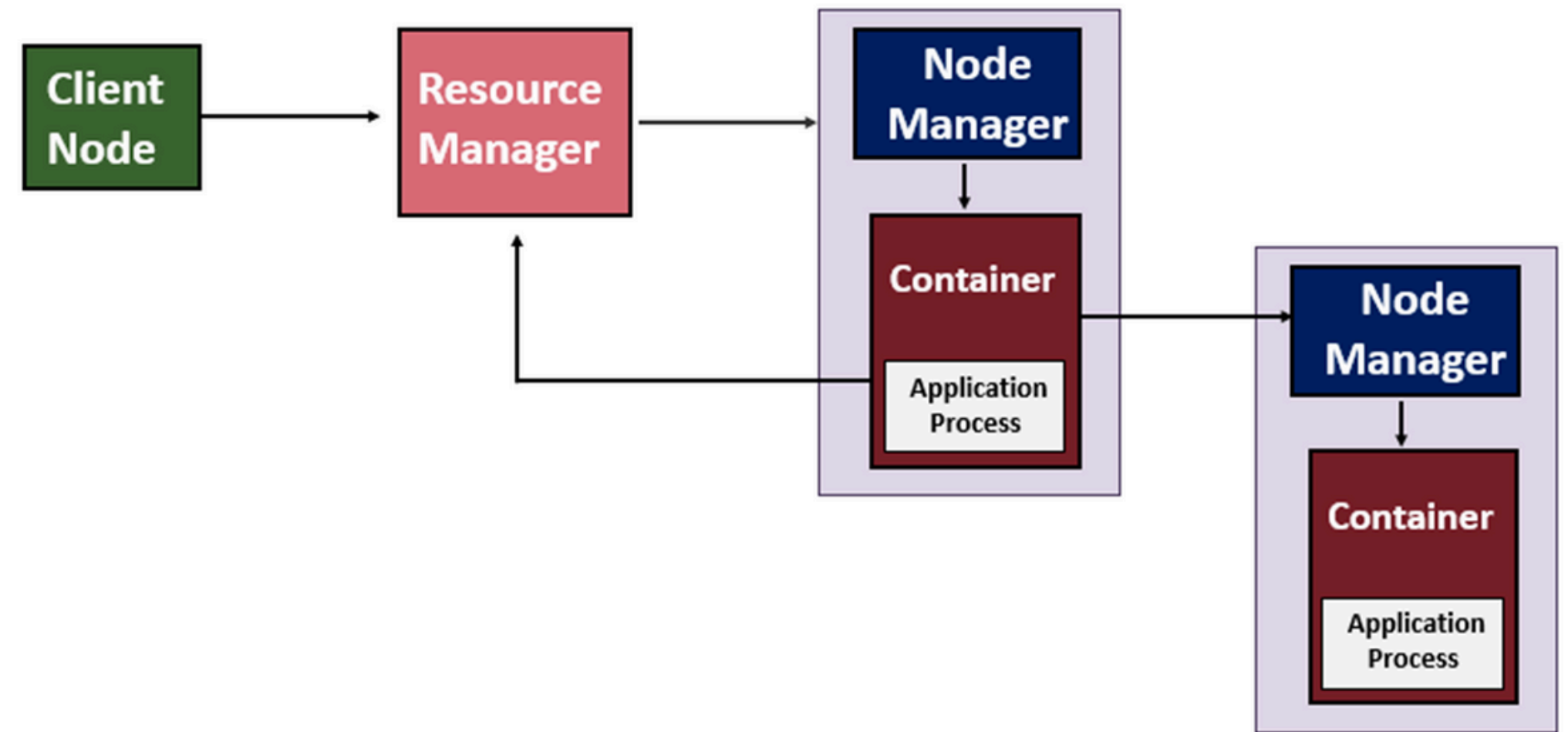
Source: <https://dataconomy.com/2014/06/hadoop-components-need-know/>



YARN Architecture

Yet Another Resource Negotiator

- Managing computing resources in clusters
 - Containers: CPU, Memory
- Application Master
 - Scheduling users' applications
 - Monitor execution of tasks and lifecycle of applications

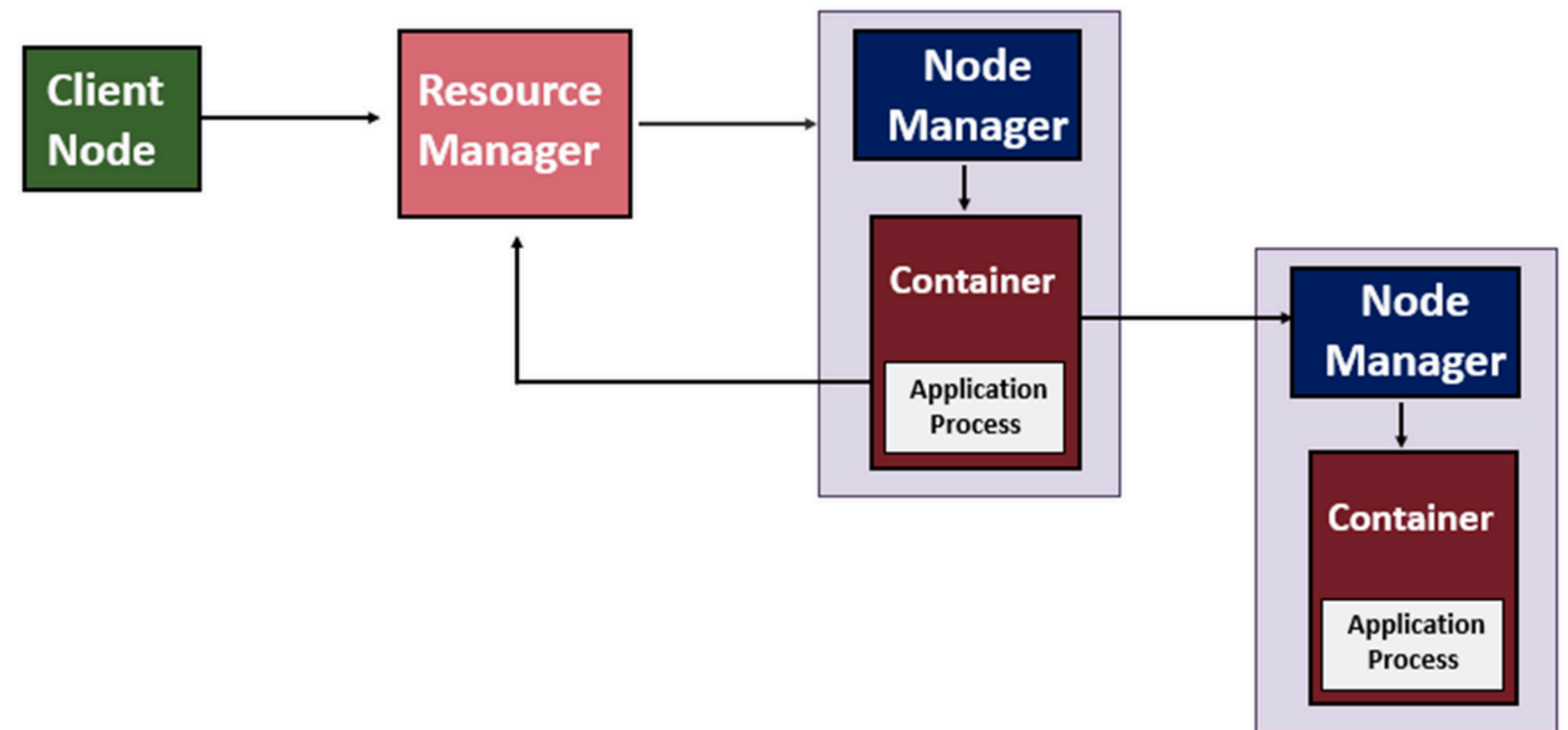


educba.com

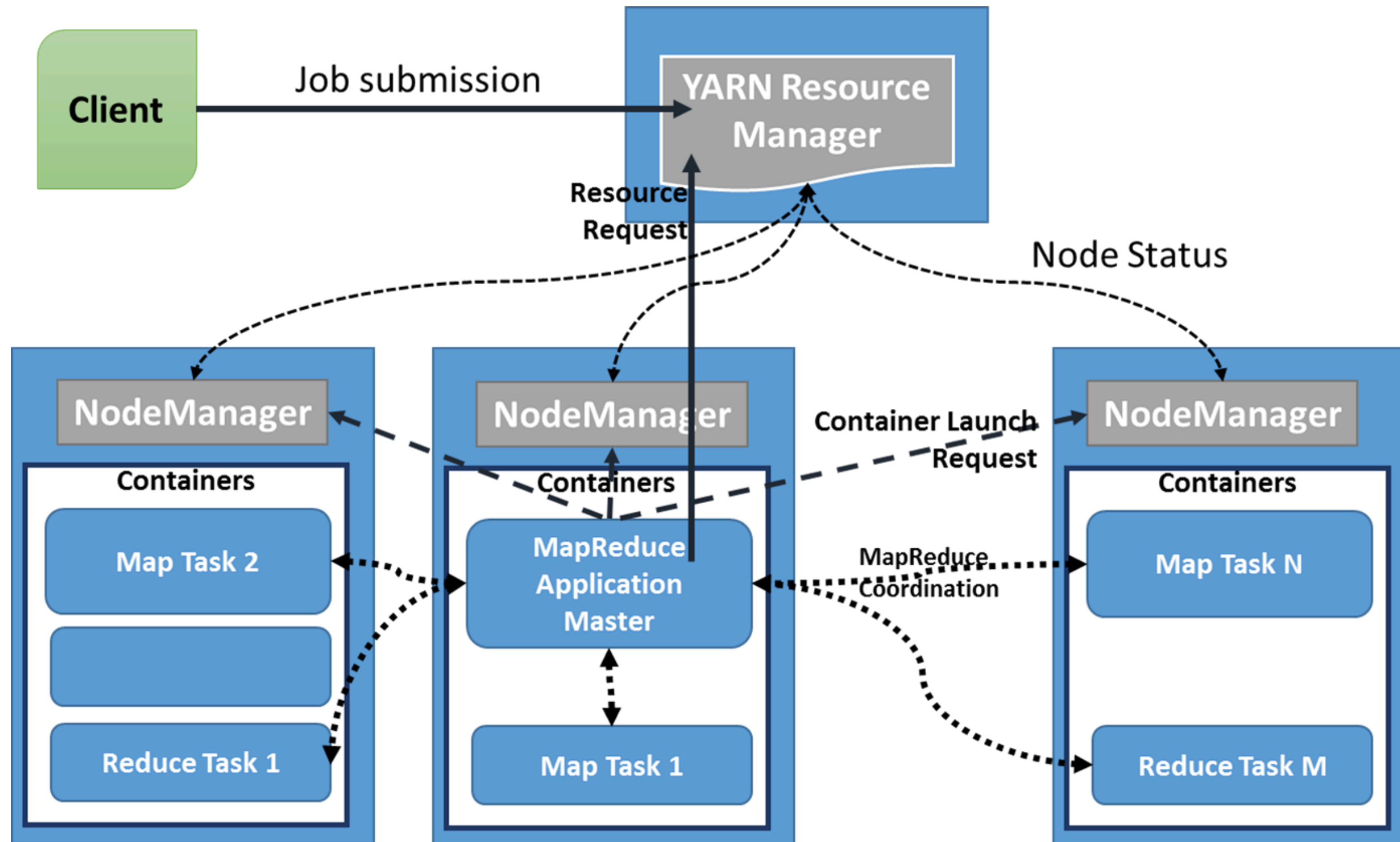
YARN Architecture

2 Core Components of YARN:

- Resource Manager
 - **Application Manager:** It manages the running of Application Master in a cluster and on the failure of the Application Master Container
 - **Scheduler:** Scheduling is performed based on the requirement of resources by the applications
- Node Manager
 - Executes task in Data Node via a container.



YARN Scheduler



YARN Application

